

Chapter 6: Image Classification with CNNs

Basics and Applications of Deep Learning

Fall 2026

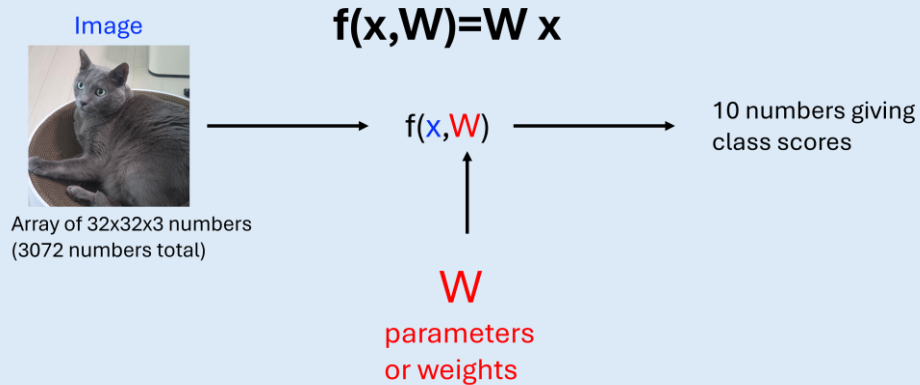
Department of Mathematical Data Science

Hanyang University ERICA

Lecturer: Sehyun Kwon



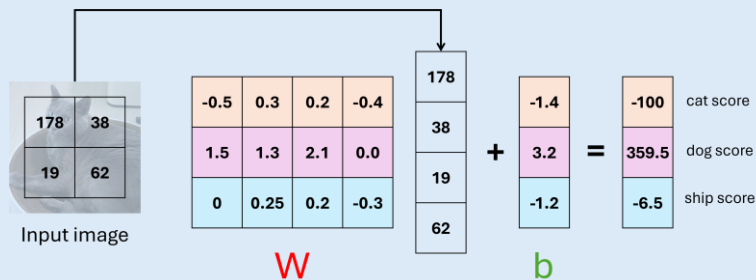
(Recap) Image classification with linear classifier



$$f(x, W) = Wx$$

Algebraic Viewpoint

$$f(x, W) = Wx$$



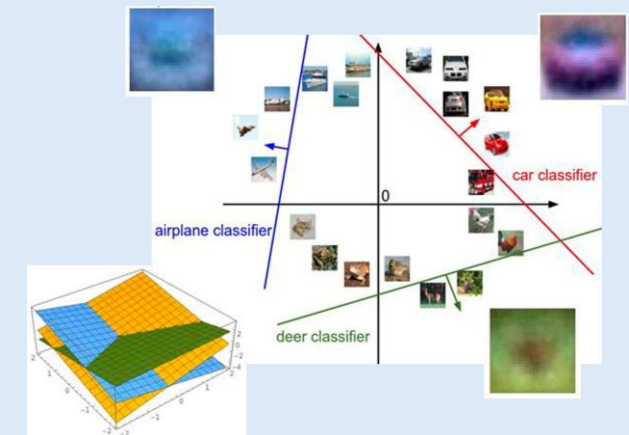
Visual Viewpoint

One template per class



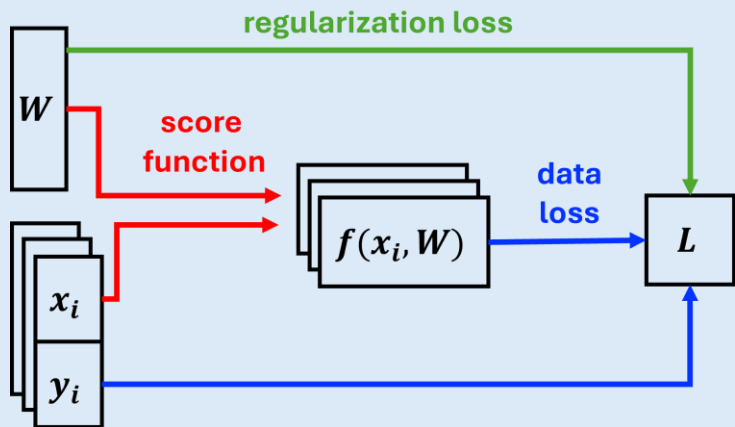
Geometric Viewpoint

Hyperplanes cutting up space



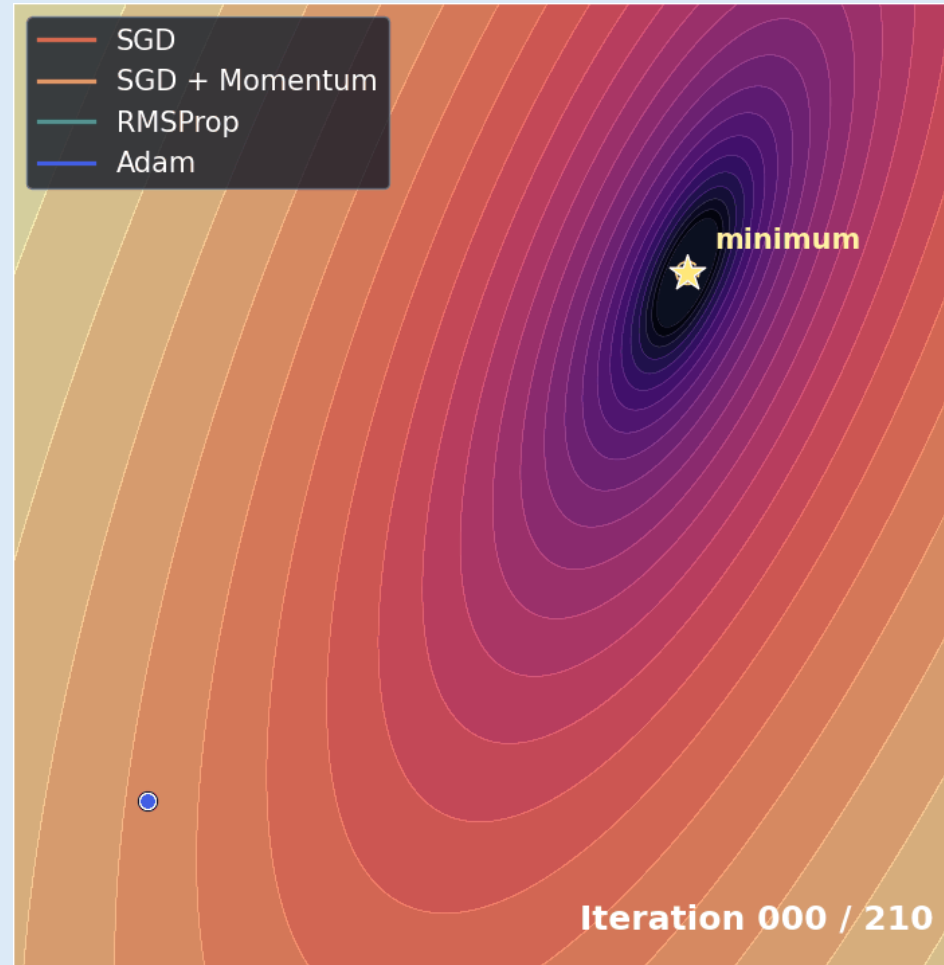
(Recap) Loss Function

- We have some dataset of (x, y)
- We have a score function: $s = f(x; W) = Wx$
- We have a loss function: $L_i = -\log \left(\frac{e^{s y_i}}{\sum_j e^{s_j}} \right)$ Softmax



$$L_i = \frac{1}{N} \sum_{i=1}^N L_i + R(W) \quad \text{Full loss}$$

(Recap) Optimization



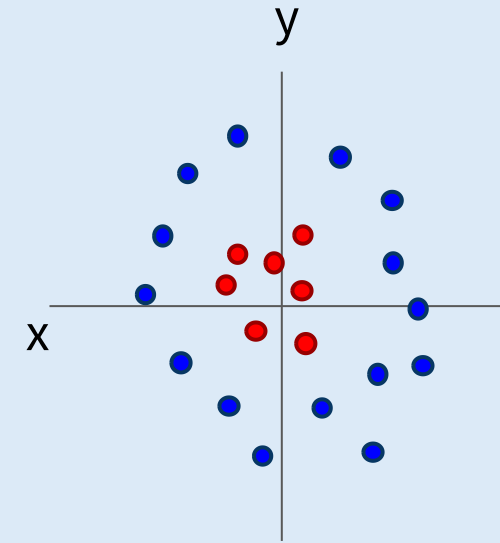
Problem: Linear classifiers are NOT very powerful

Visual Viewpoint



Linear classifiers learn
one template per class

Geometric Viewpoint

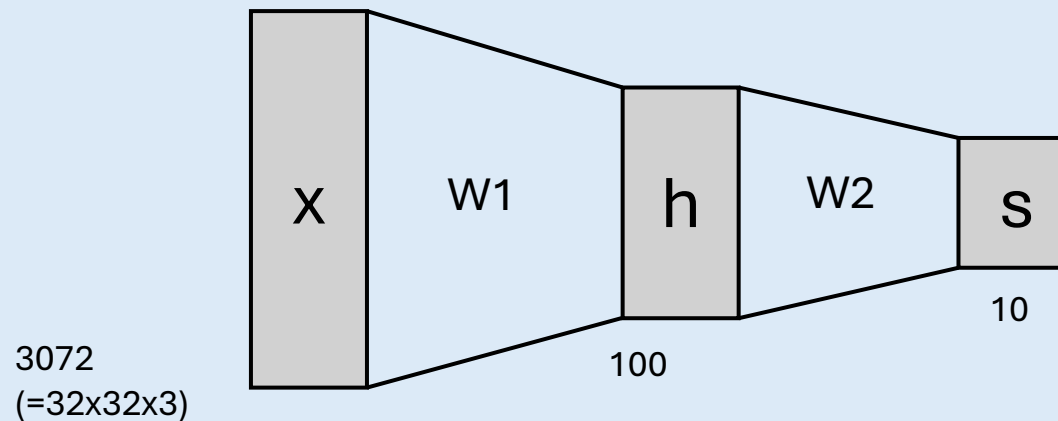


Linear classifiers can only draw
linear decision boundaries

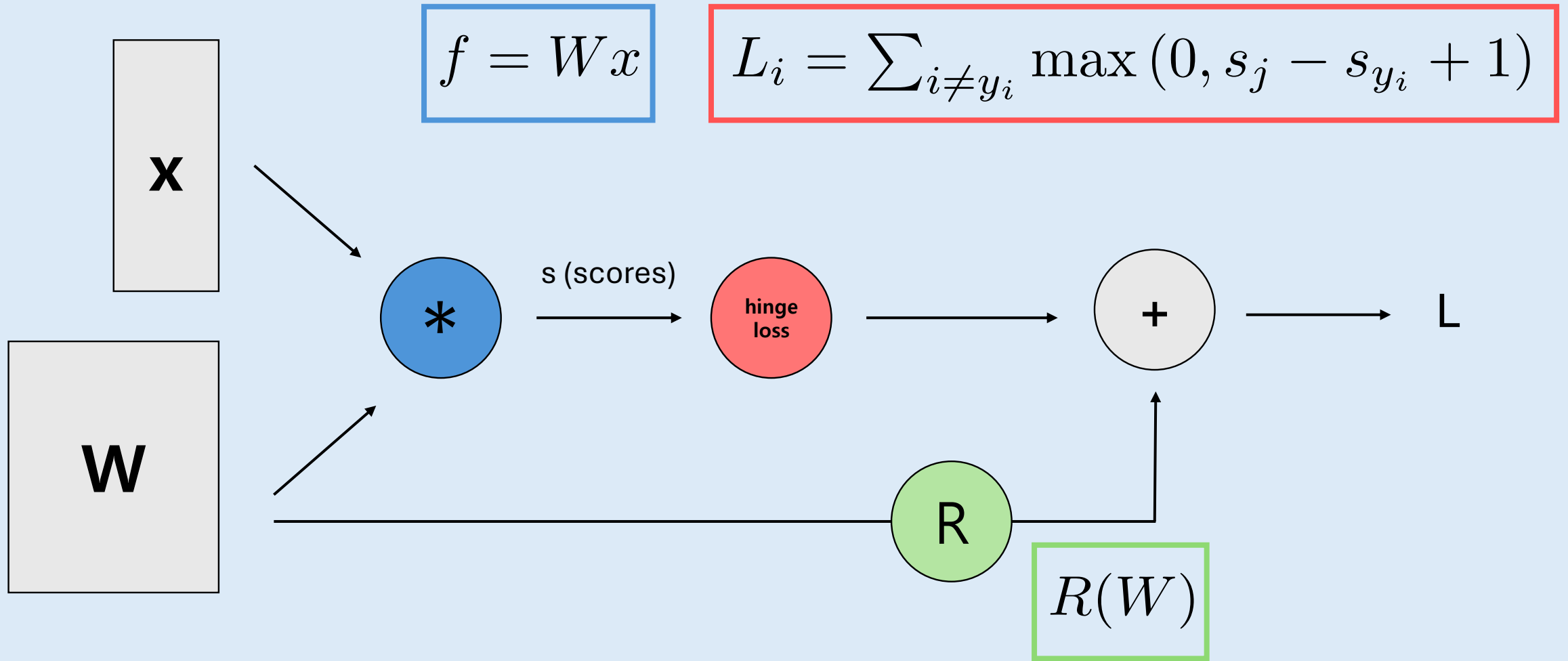
Last time: Neural Networks

Linear score function: $f = Wx$

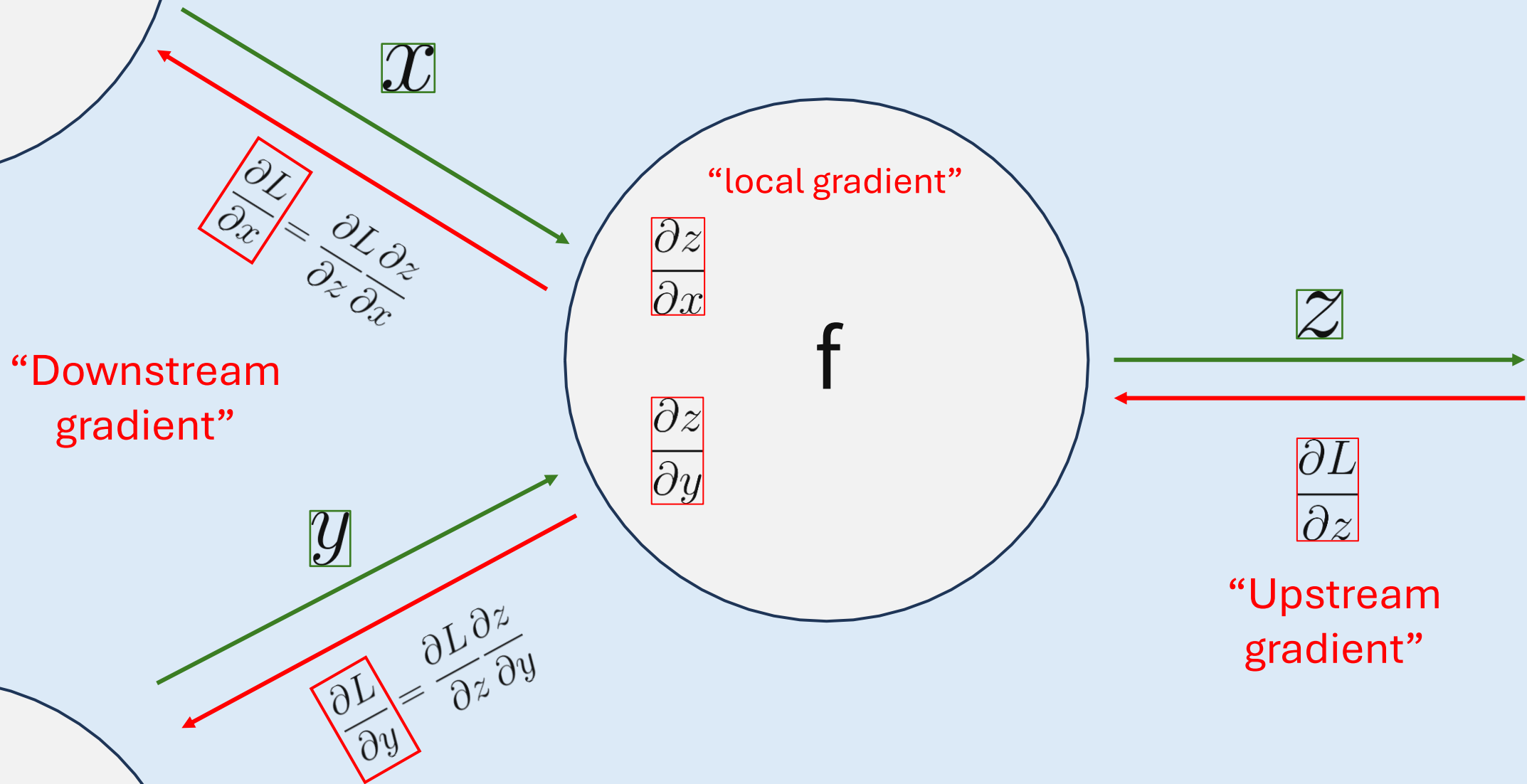
2-layer Neural Network: $f = W_2 \max(0, W_1 x)$



Last time: Computation Graph



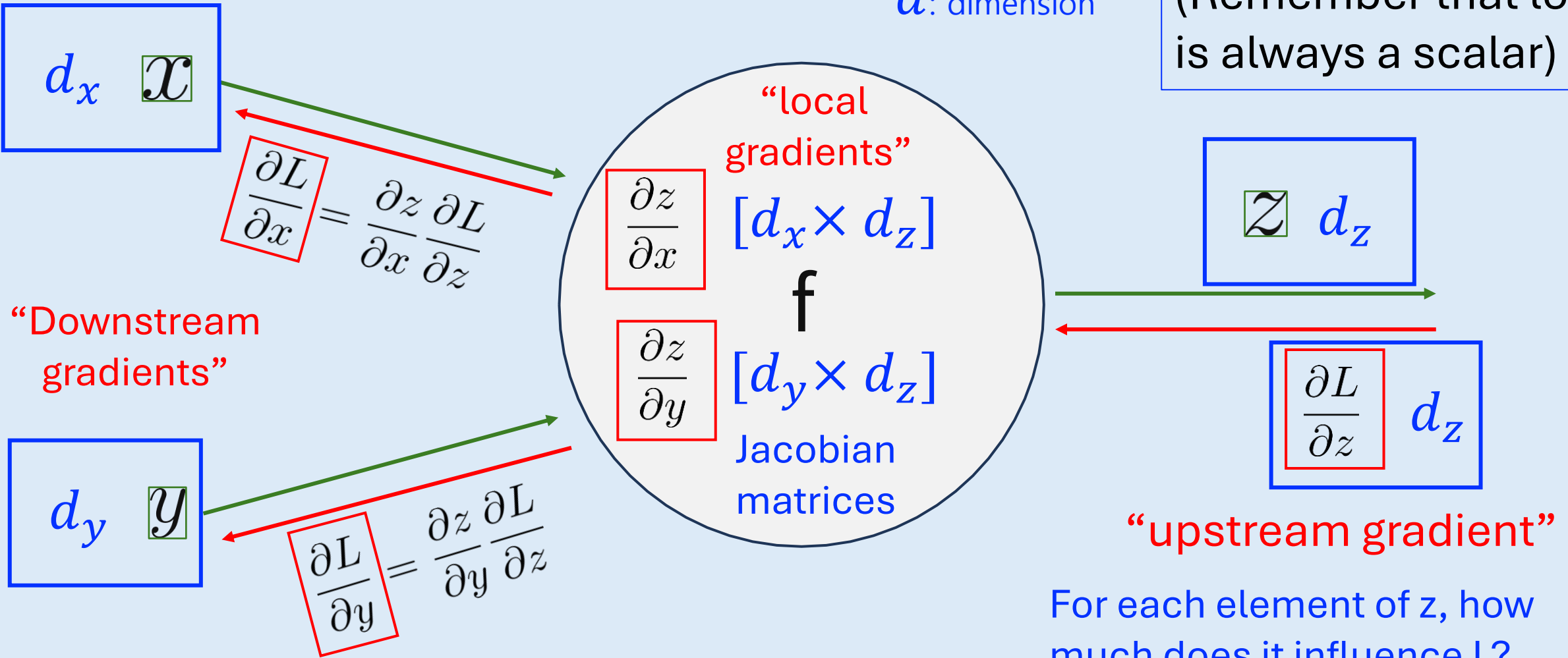
Last time: Backpropagation



Last time: Backprop with vectors

d : dimension

Loss L still a scalar!
(Remember that loss is always a scalar)



For each element of z , how much does it influence L ?

Last time: backprop with matrices (or tensors)

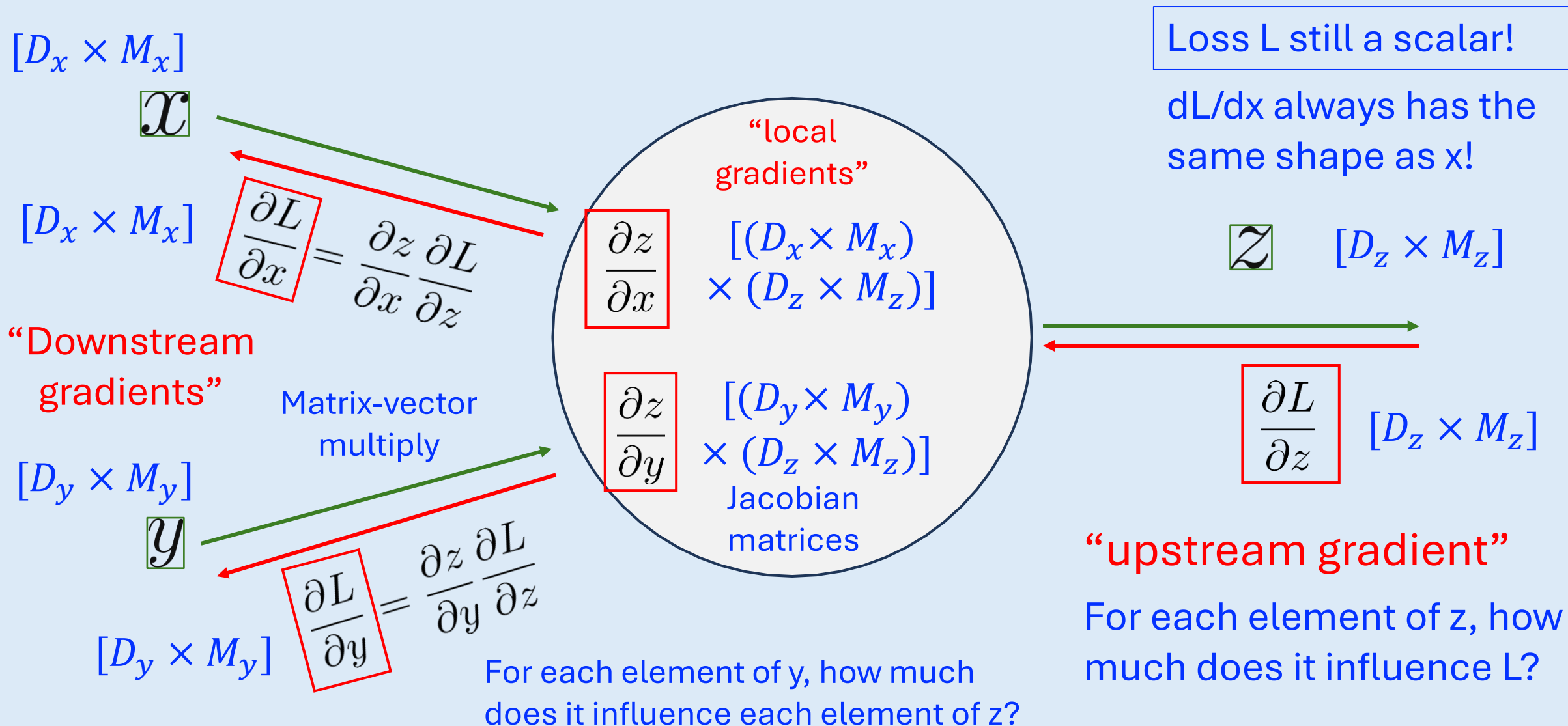


Image classification: A core task in computer vision



(Assume given a set of labels)
{dog, cat, truck, plane, ...}



cat

dog

bird

deer

truck

Pixel space

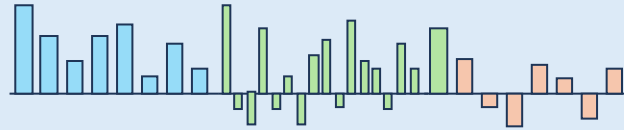


$$f(x) = Wx$$

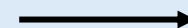
Class
scores



Image features

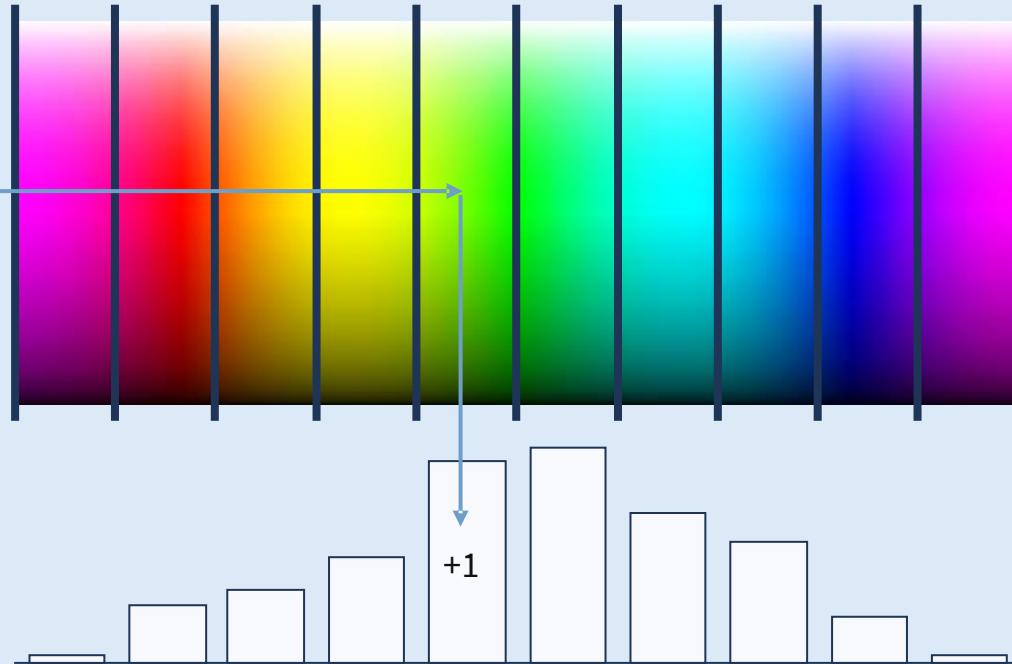


$$f(x) = Wx$$

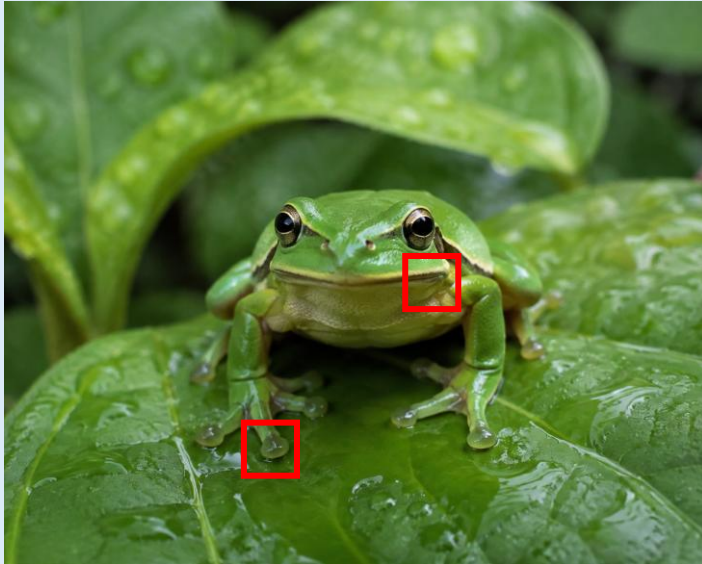


Class
scores

Example: Color Histogram



Example: Histogram of Oriented Gradients (HoG)



Divide the image into 8×8 -pixel regions.
Within each region, quantize the edge
directions into 9 bins.



Our 1400×1120 frog image is divided into
 175×140 cells. Each cell contains 9
values, so the feature vector has
 $175 \times 140 \times 9 = 220,500$ values.

Image Features

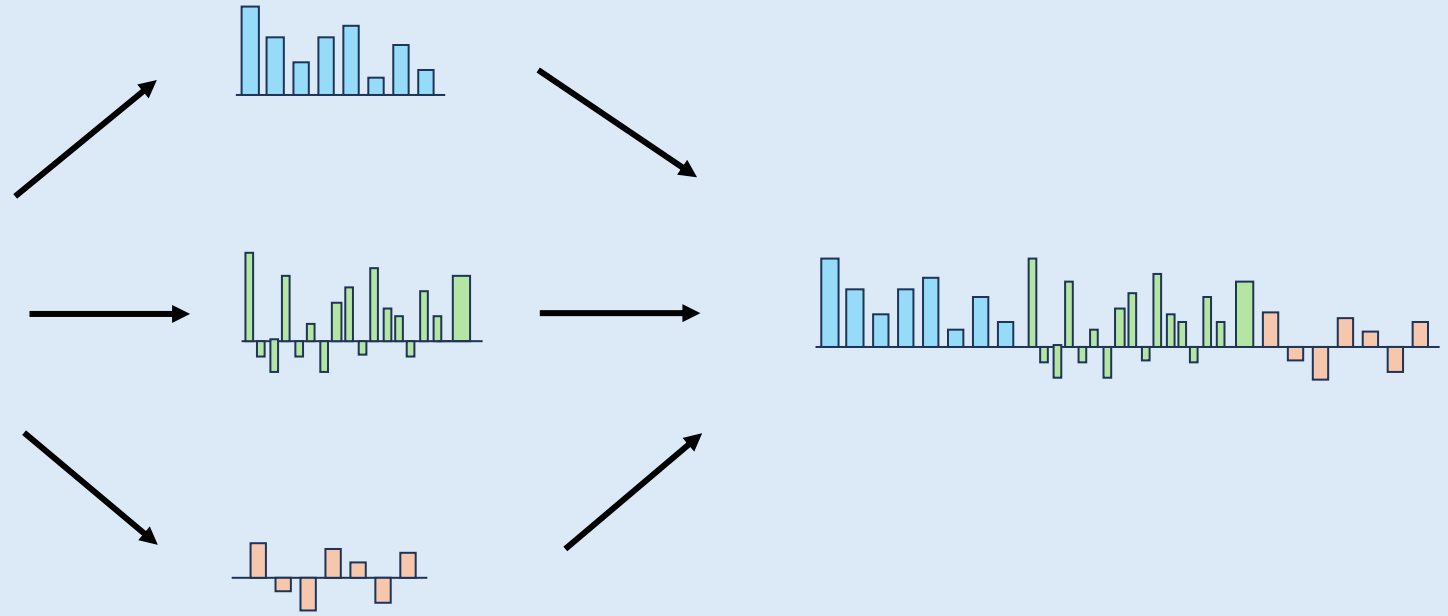
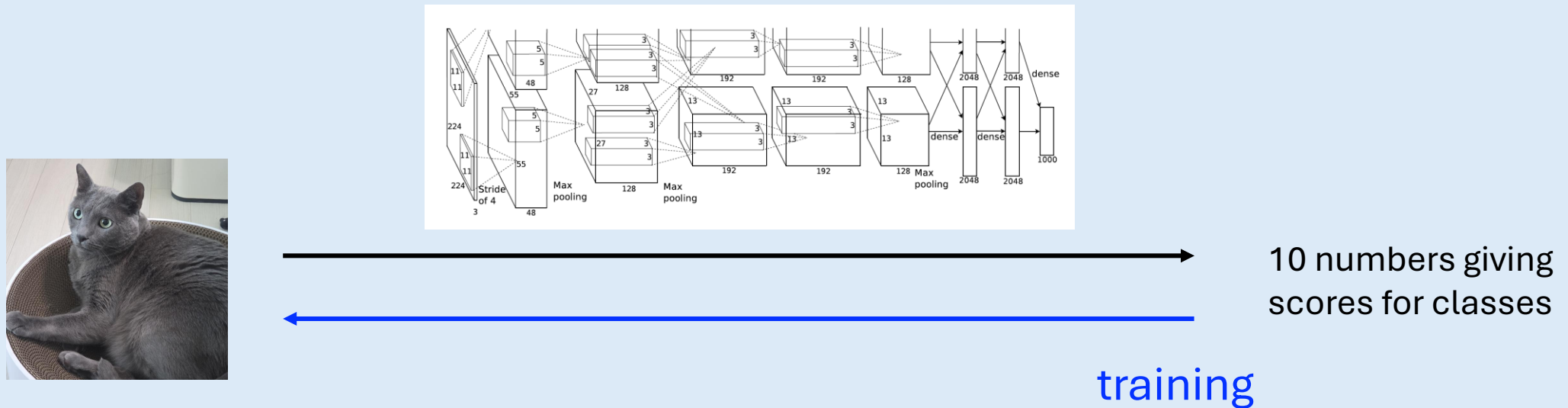
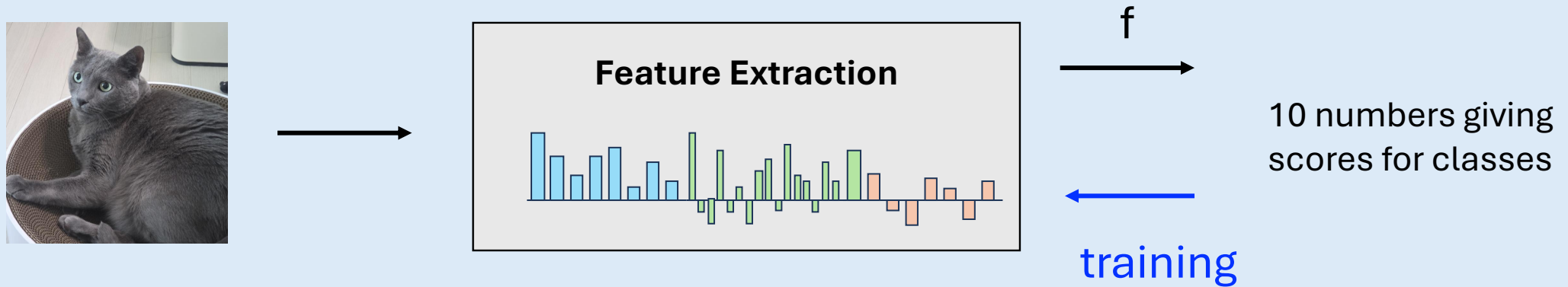


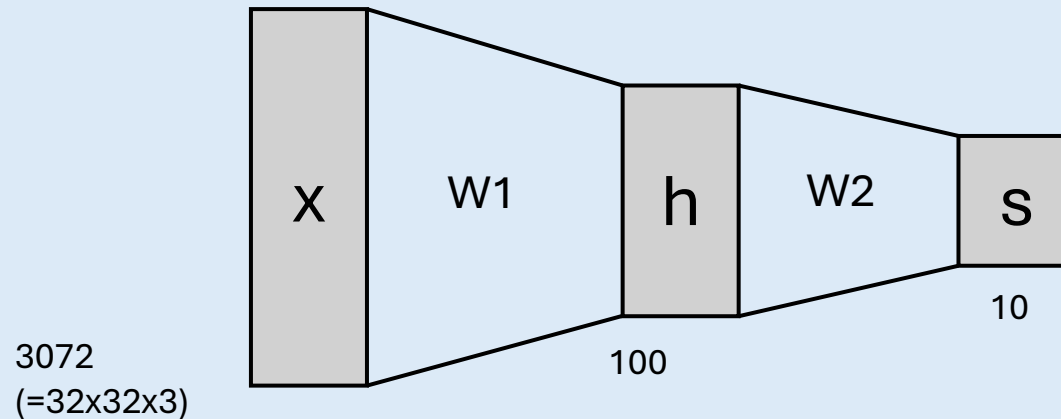
Image features vs. ConvNets



Last time: Neural networks

Linear score function: $f = Wx$

2-layer Neural Network: $f = W_2 \max(0, W_1 x)$



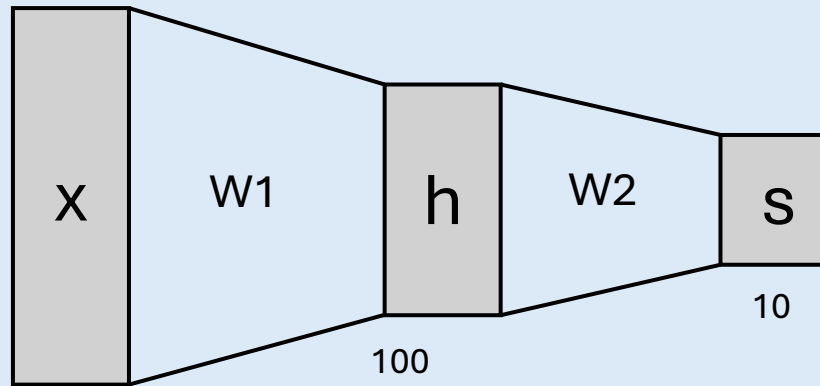
Last time: Neural networks

Linear score function: $f = Wx$

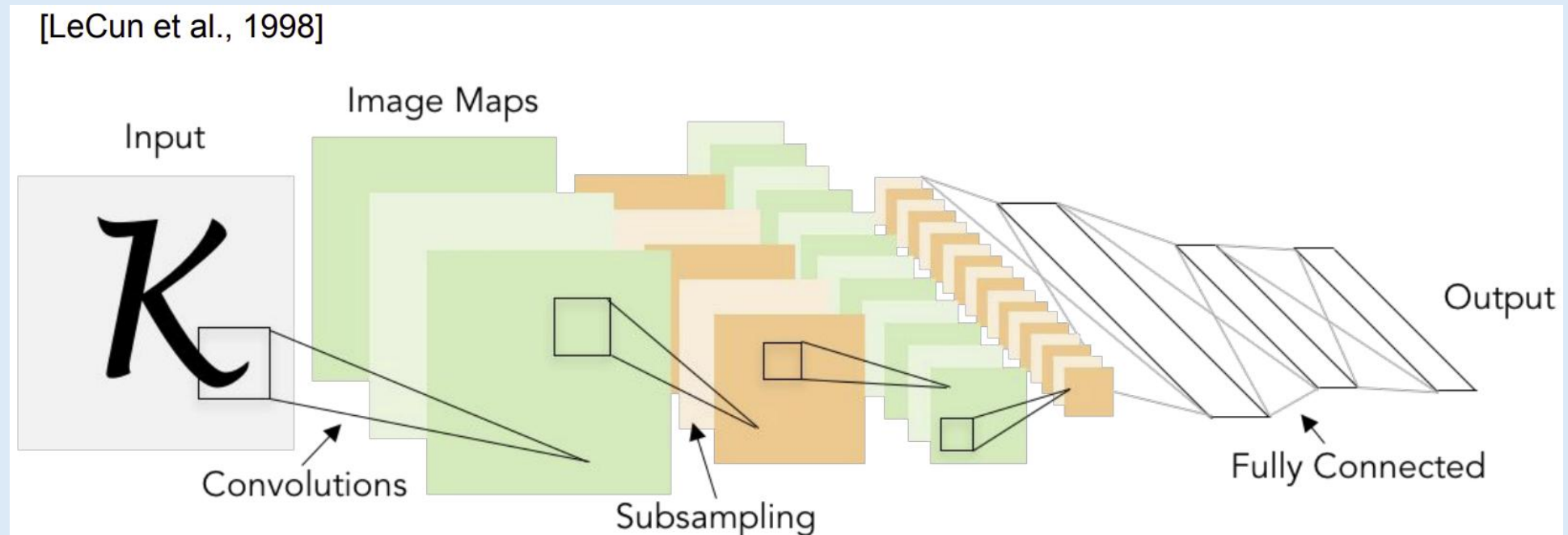
2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

Problem: The spatial
structure of images is
destroyed!

3072
(=32x32x3)

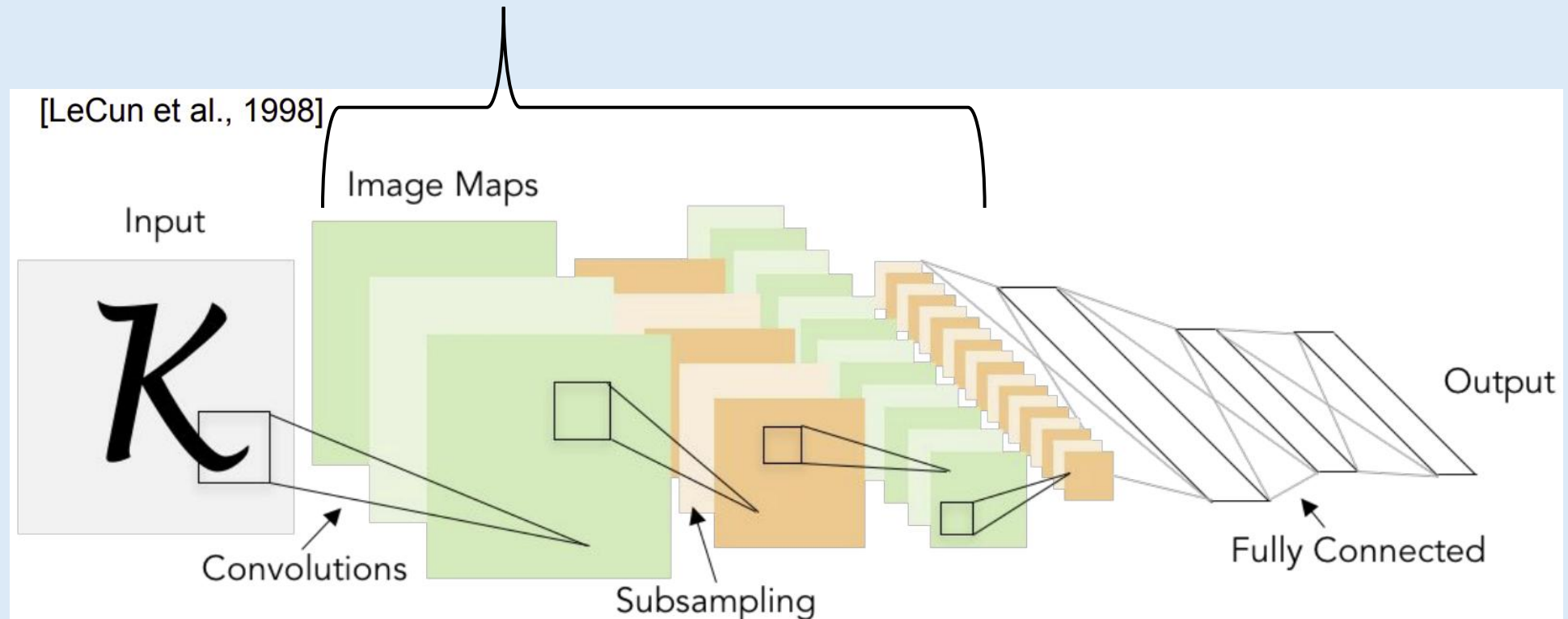


Next: Convolutional Neural Networks (CNNs)



Next: Convolutional Neural Networks (CNNs)

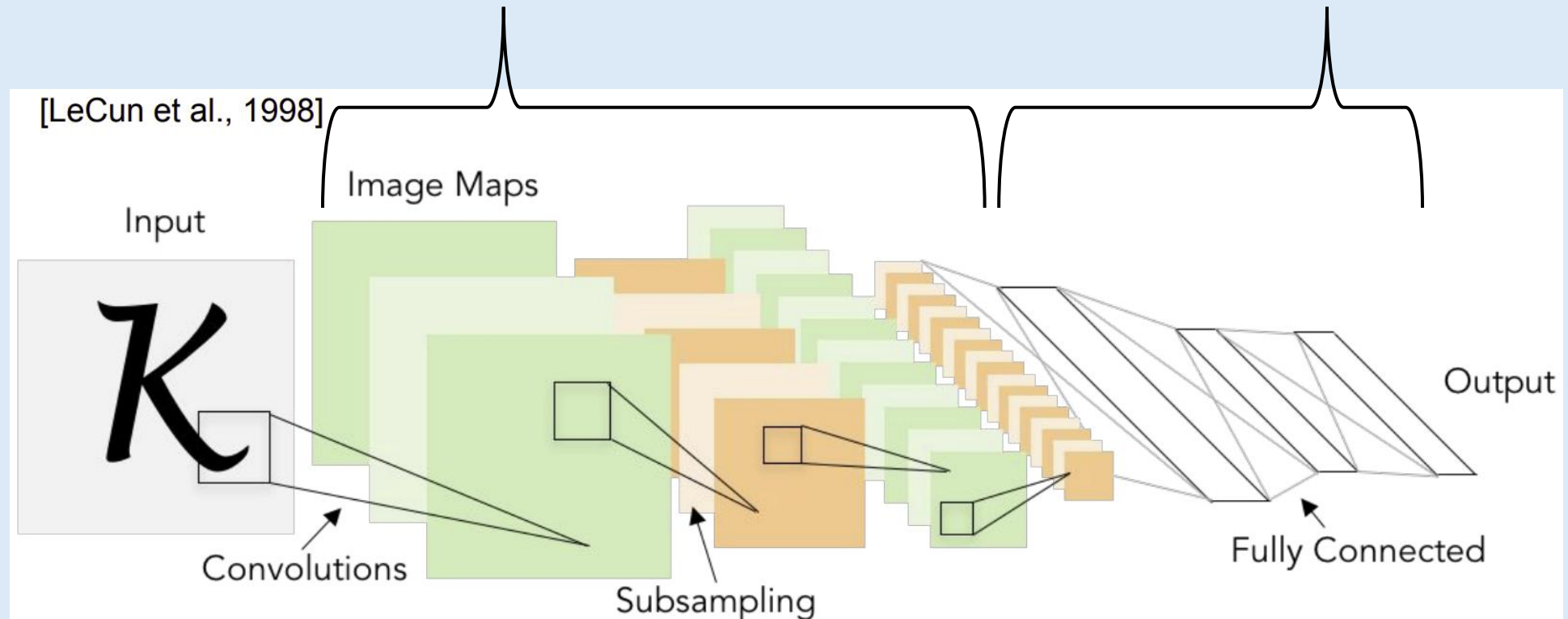
Convolution and **pooling** operators extract features while respecting 2D image structure



Next: Convolutional Neural Networks (CNNs)

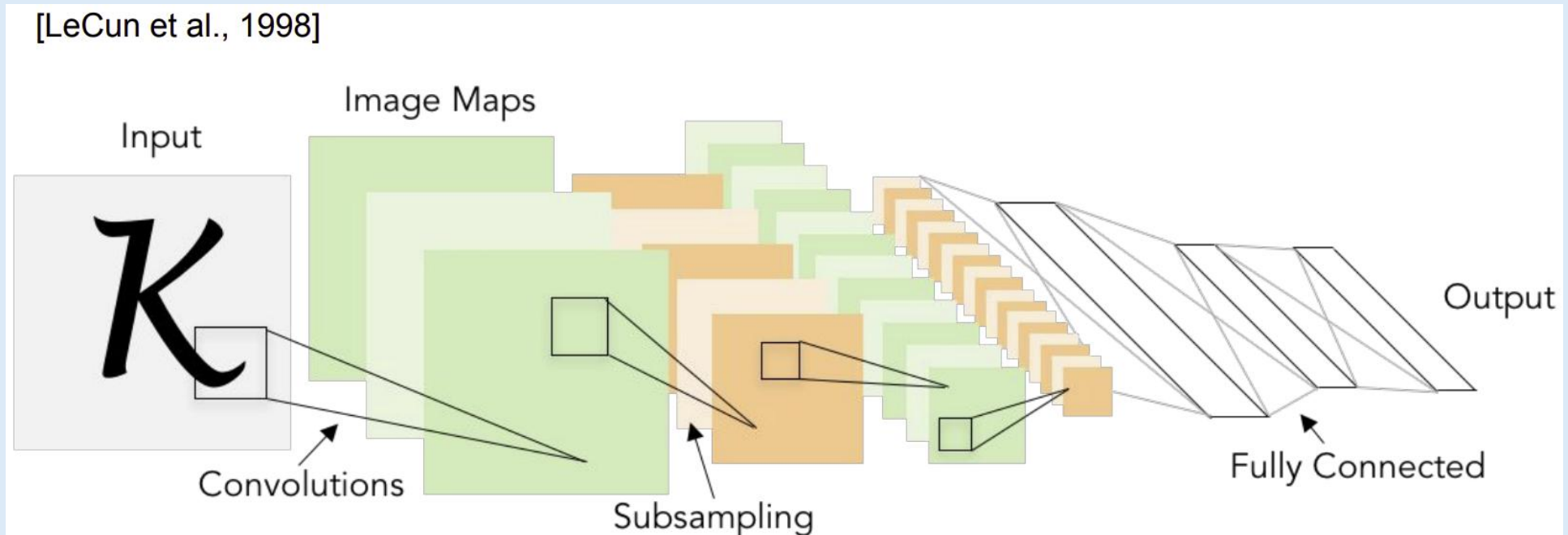
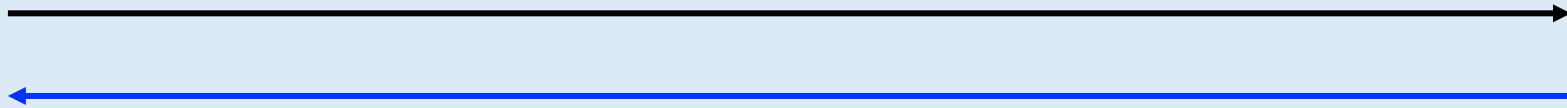
Convolution and **pooling** operators extract features while respecting 2D image structure

Fully-Connected layers form an MLP at the end to predict scores

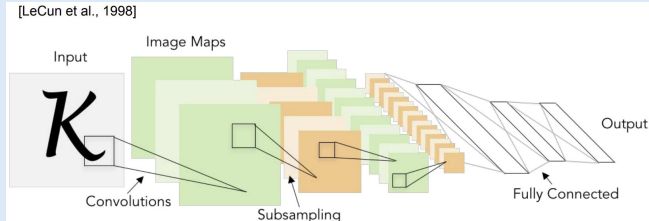


Next: Convolutional Neural Networks (CNNs)

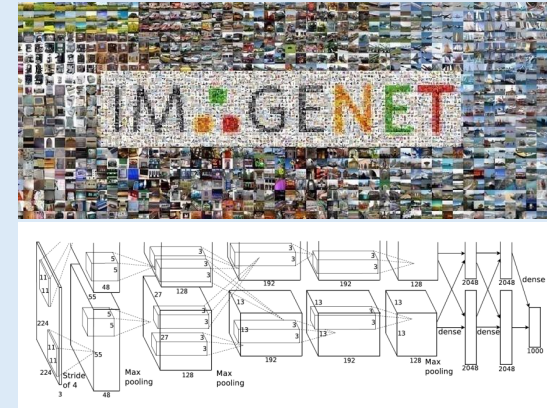
Trained **end to end** with backprop + gradient descent



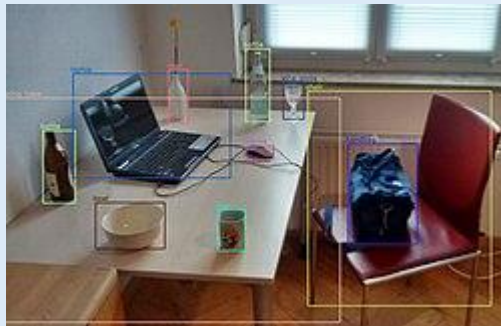
History of CNNs



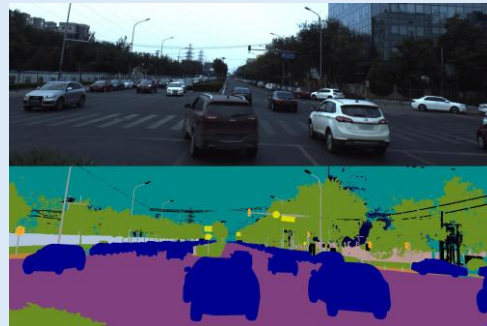
LeNet (1998): Document recognition



AlexNet (2012): ImageNet challenge



Object detection



Segmentation



A computer screen with a Windows message about Microsoft license terms.

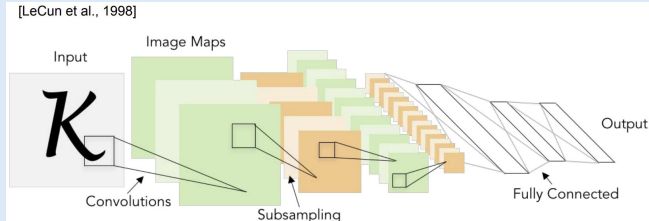


A hand holds up a can of Coors Light in front of an outdoor scene with a dog on a porch.

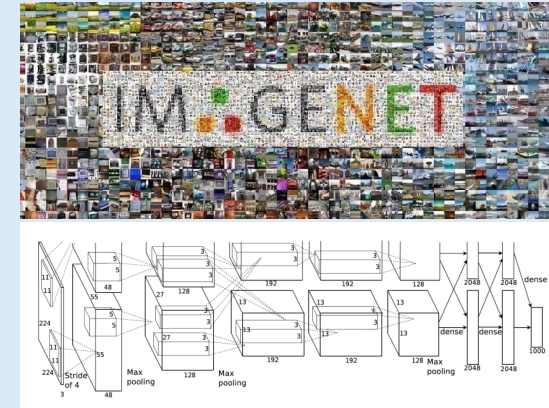
Captioning

CNNs dominate **all vision tasks** (~2012 – 2020)

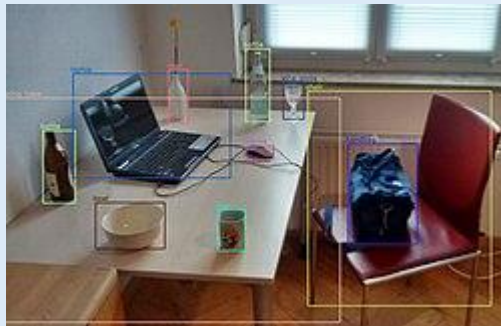
History of CNNs



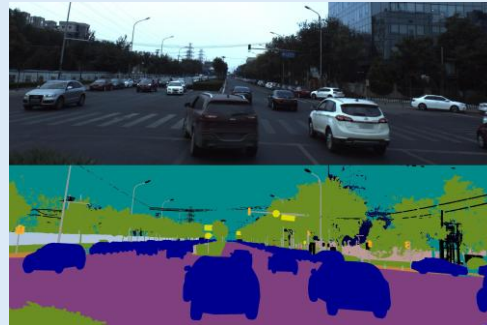
LeNet (1998): Document recognition



AlexNet (2012): ImageNet challenge



Object detection



Segmentation



A computer screen with a Windows message about Microsoft license terms.



A hand holds up a can of Coors Light in front of an outdoor scene with a dog on a porch.

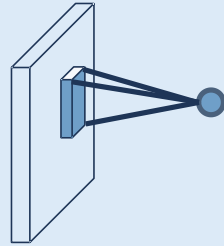
Captioning

CNNs dominate **all vision tasks** (~2012 – 2020) **before Transformers!**

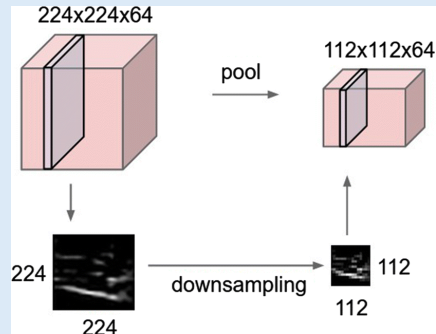
Convolutional Neural Networks (CNNs)

Today, we will study

- Convolutional layer:



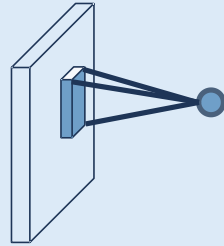
- Pooling layer:



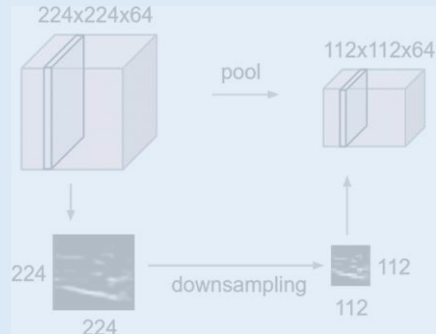
Convolutional Neural Networks (CNNs)

Today, we will study

- Convolutional layer:



- Pooling layer:



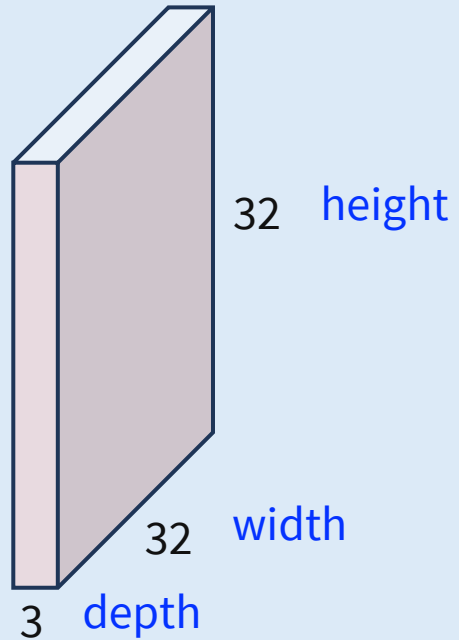
Convolutional layer

In fully connected layer, $32 \times 32 \times 3$ image should be flattened to 3072×1 .
But it causes distortion of the spatial structure.

Convolutional layer: preserving spatial structure

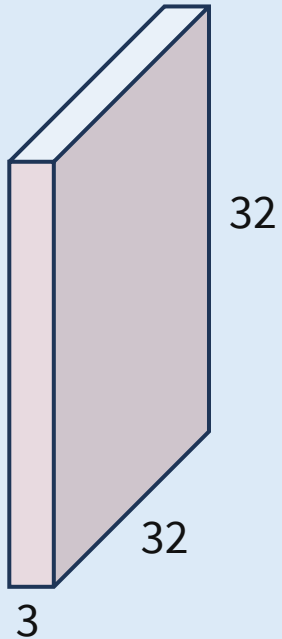
Convolutional layer

32x32x3 image



Convolutional layer

32x32x3 image



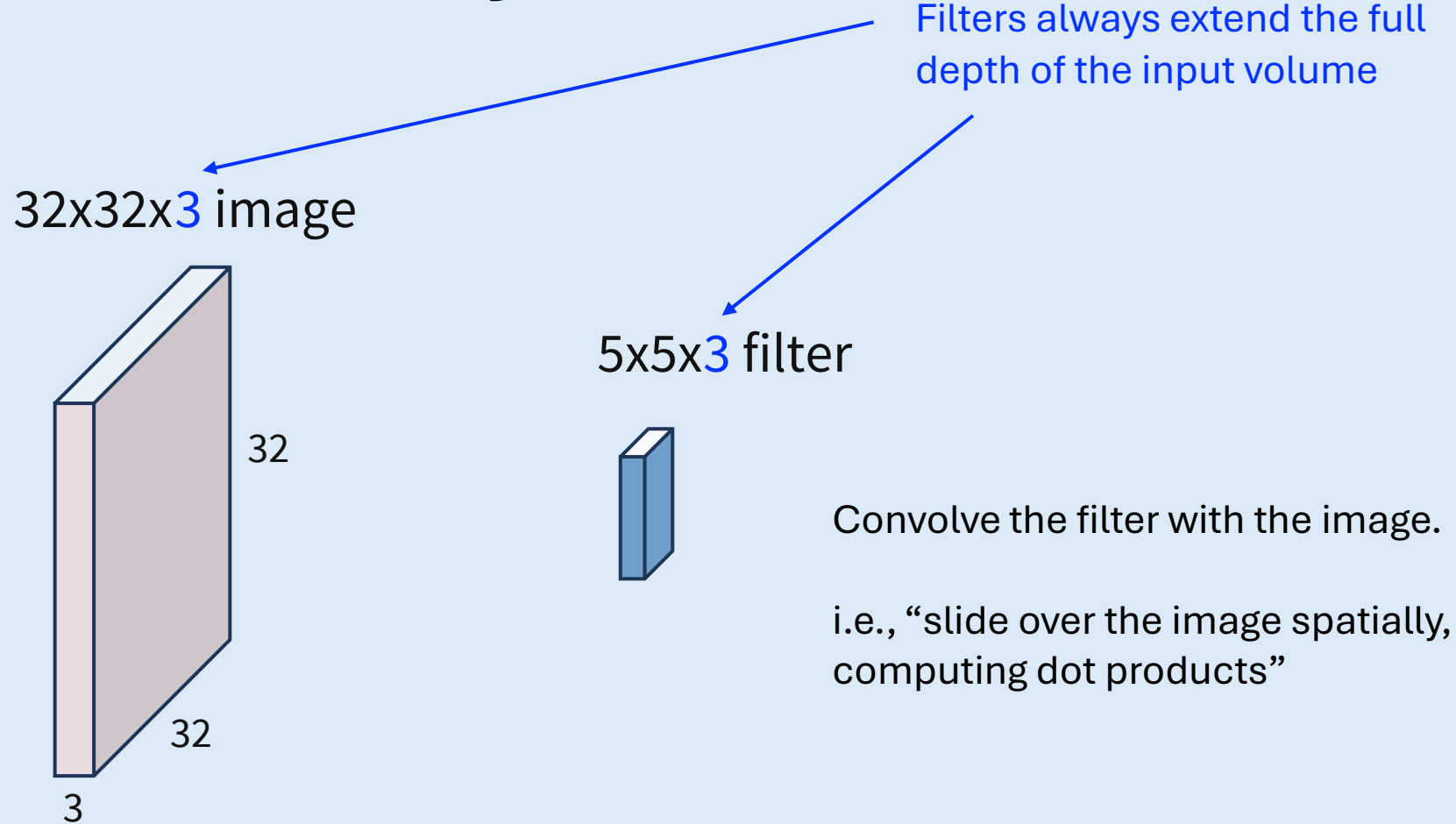
5x5x3 filter



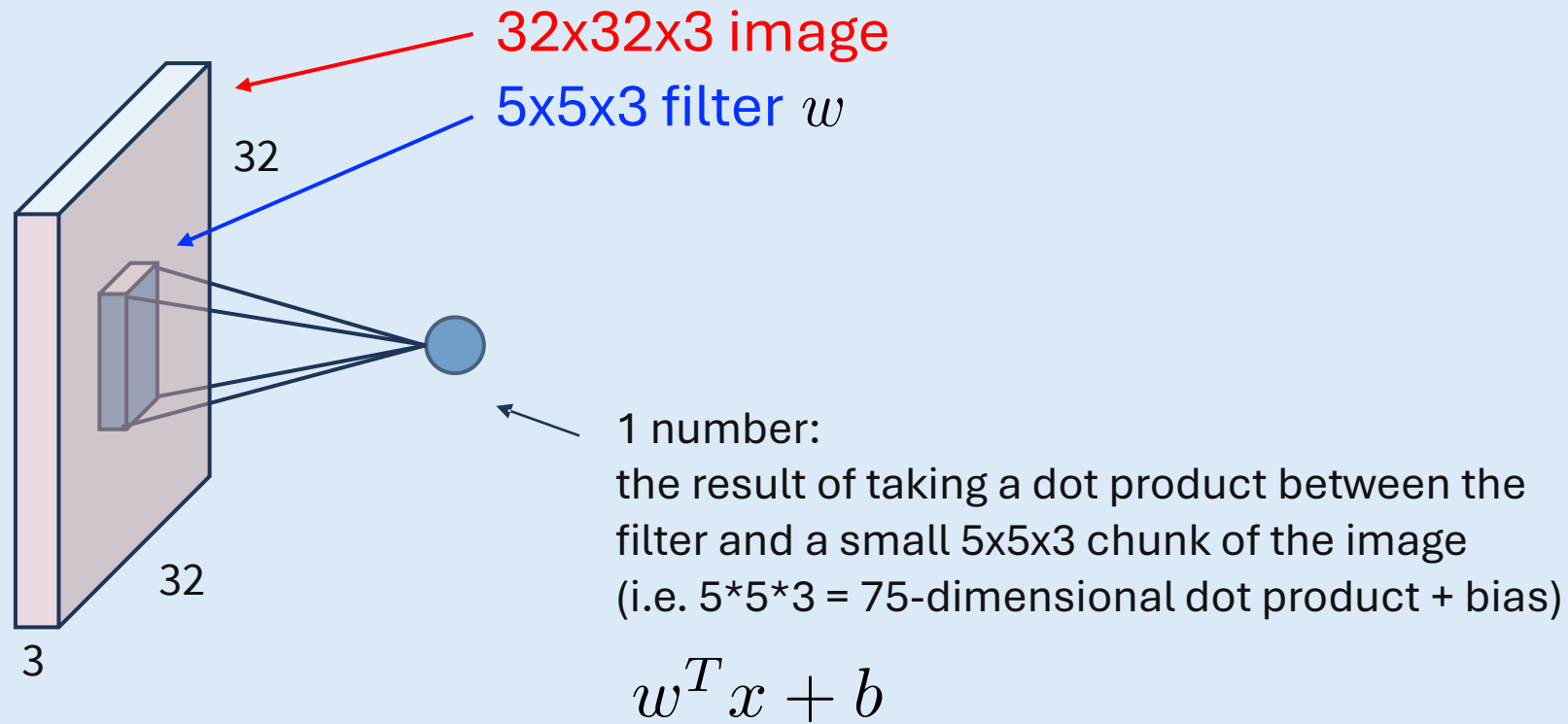
Convolve the filter with the image.

i.e., “slide over the image spatially,
computing dot products”

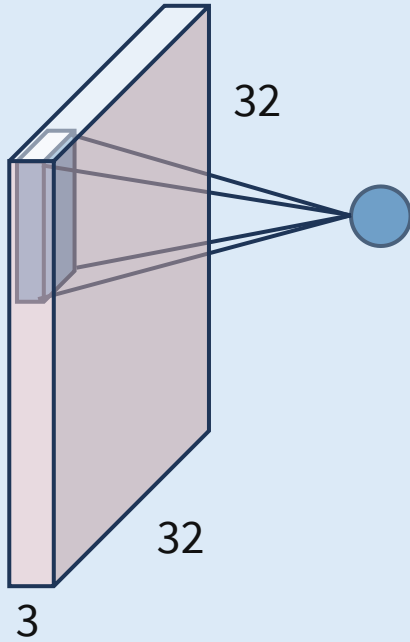
Convolutional layer



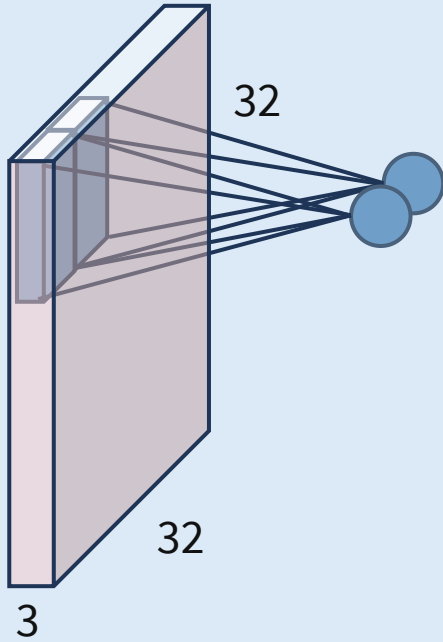
Convolutional layer



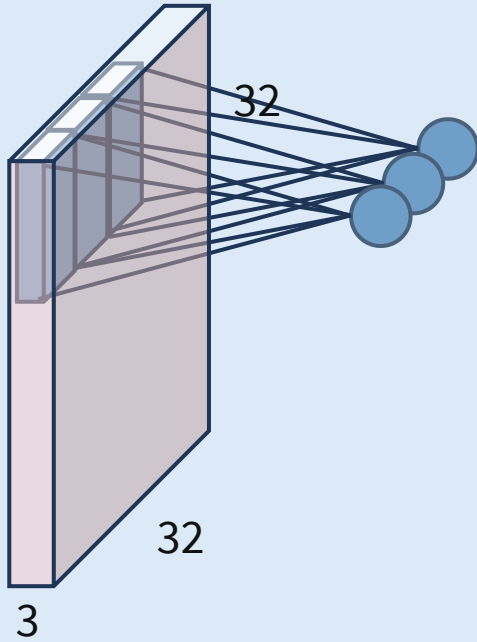
Convolutional layer



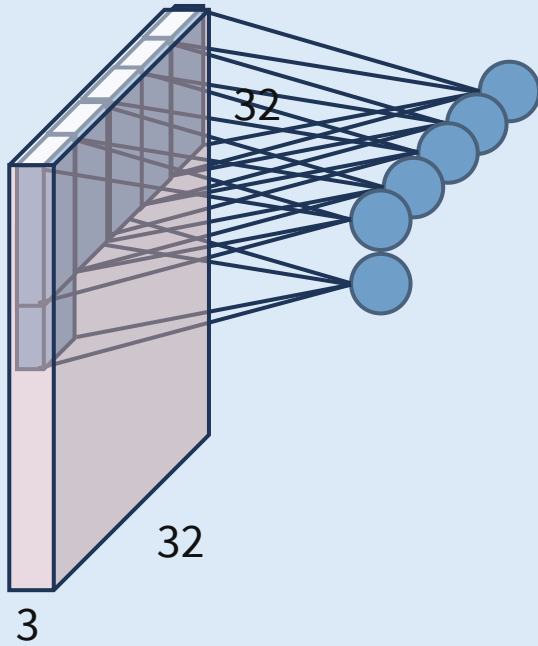
Convolutional layer



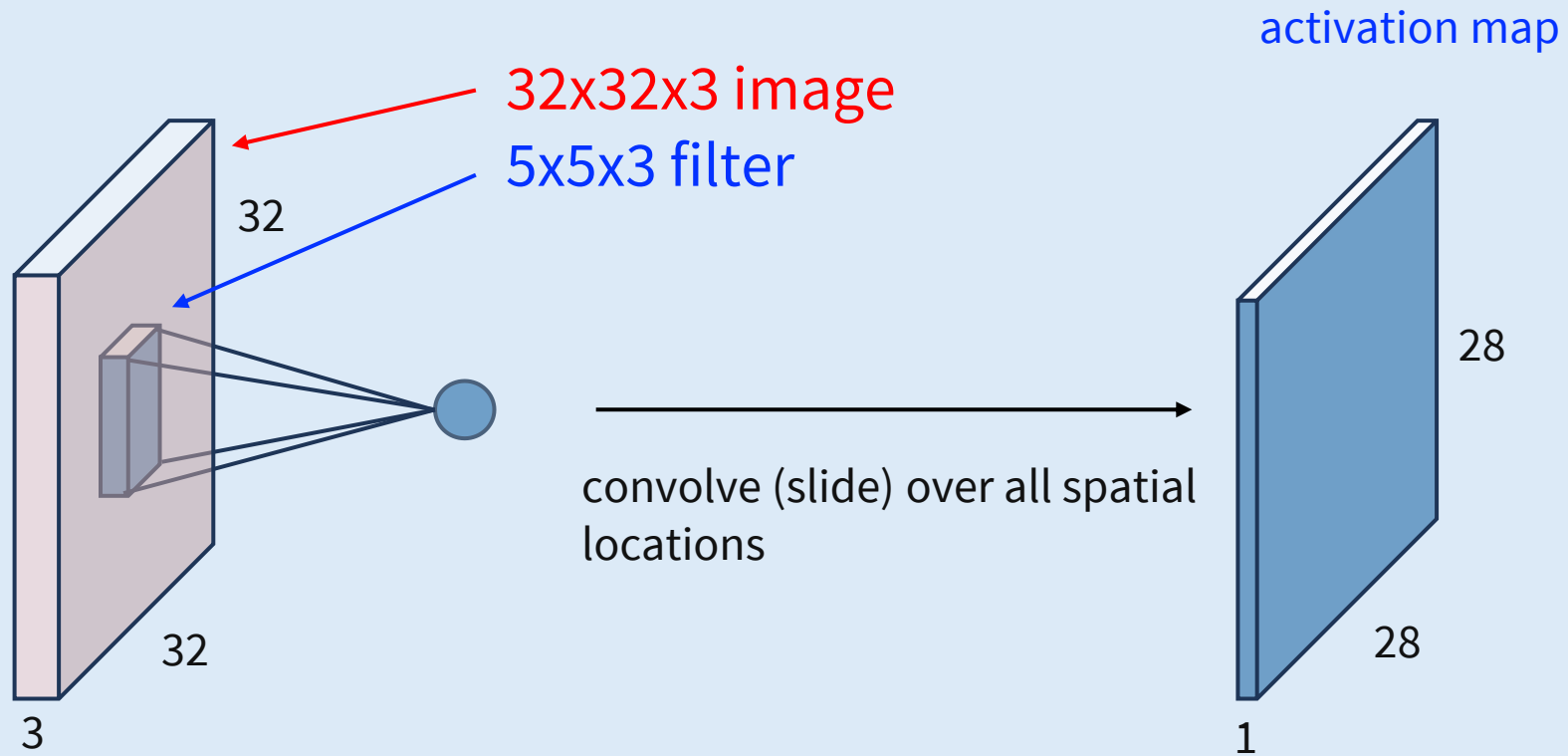
Convolutional layer



Convolutional layer

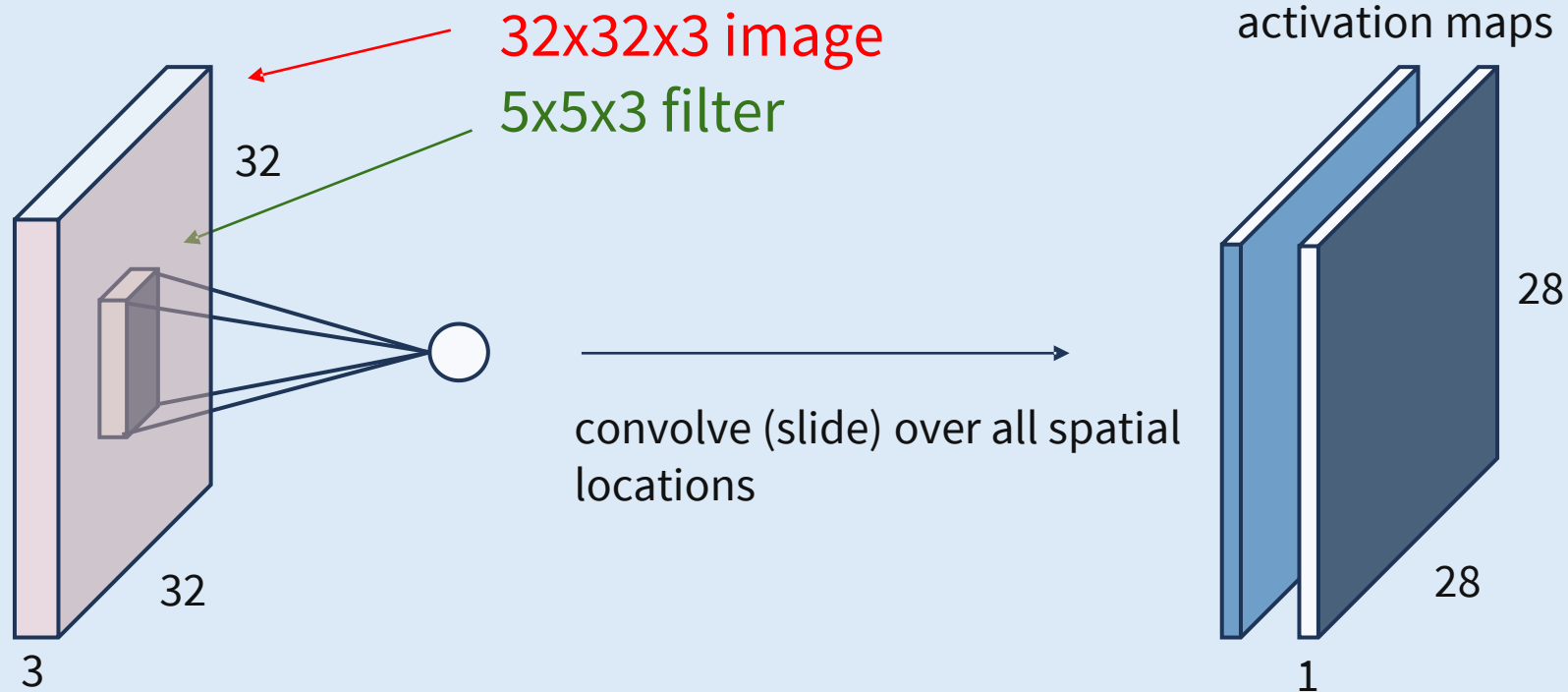


Convolutional layer

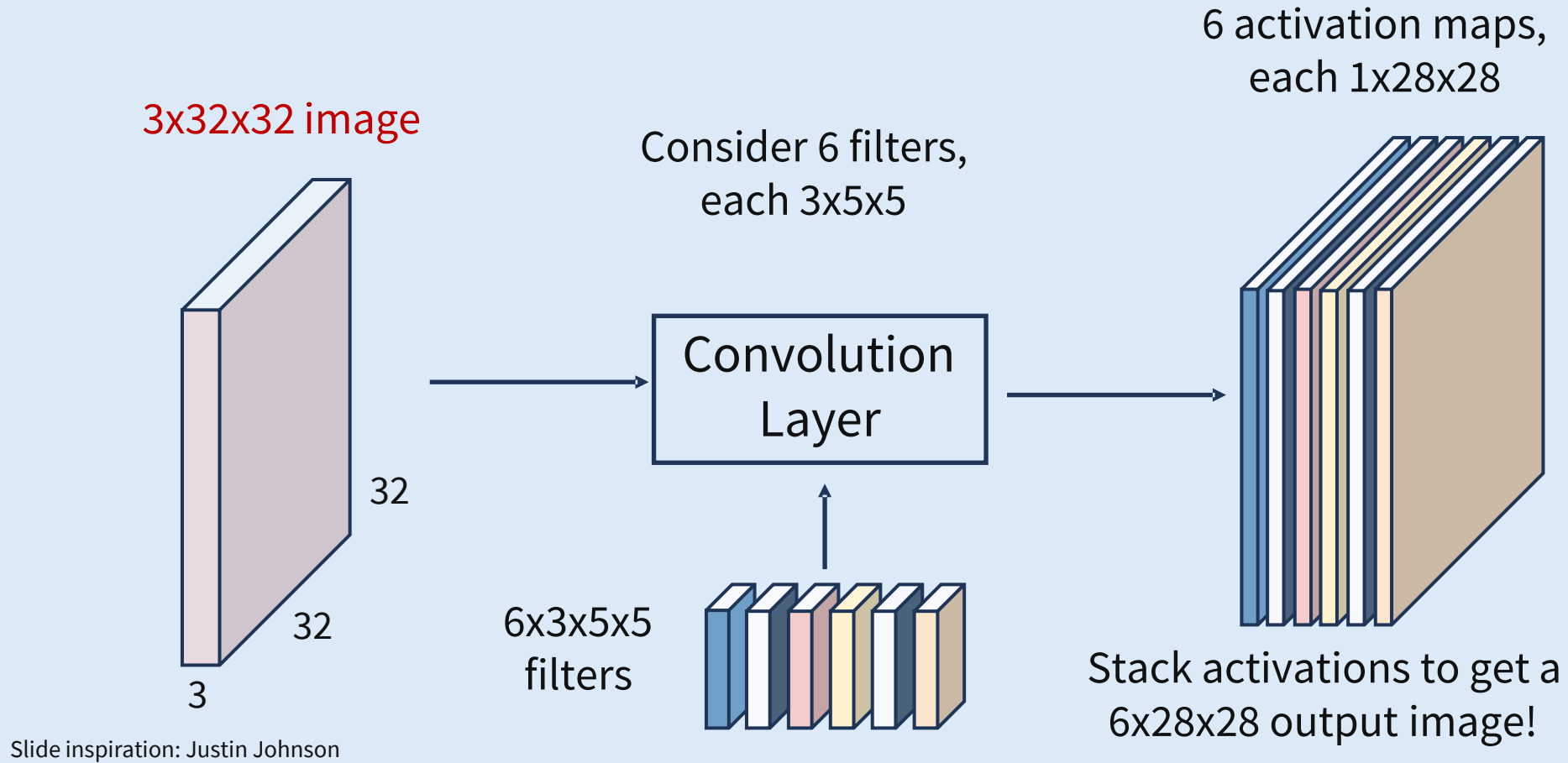


Convolutional layer

consider a second, **green** filter

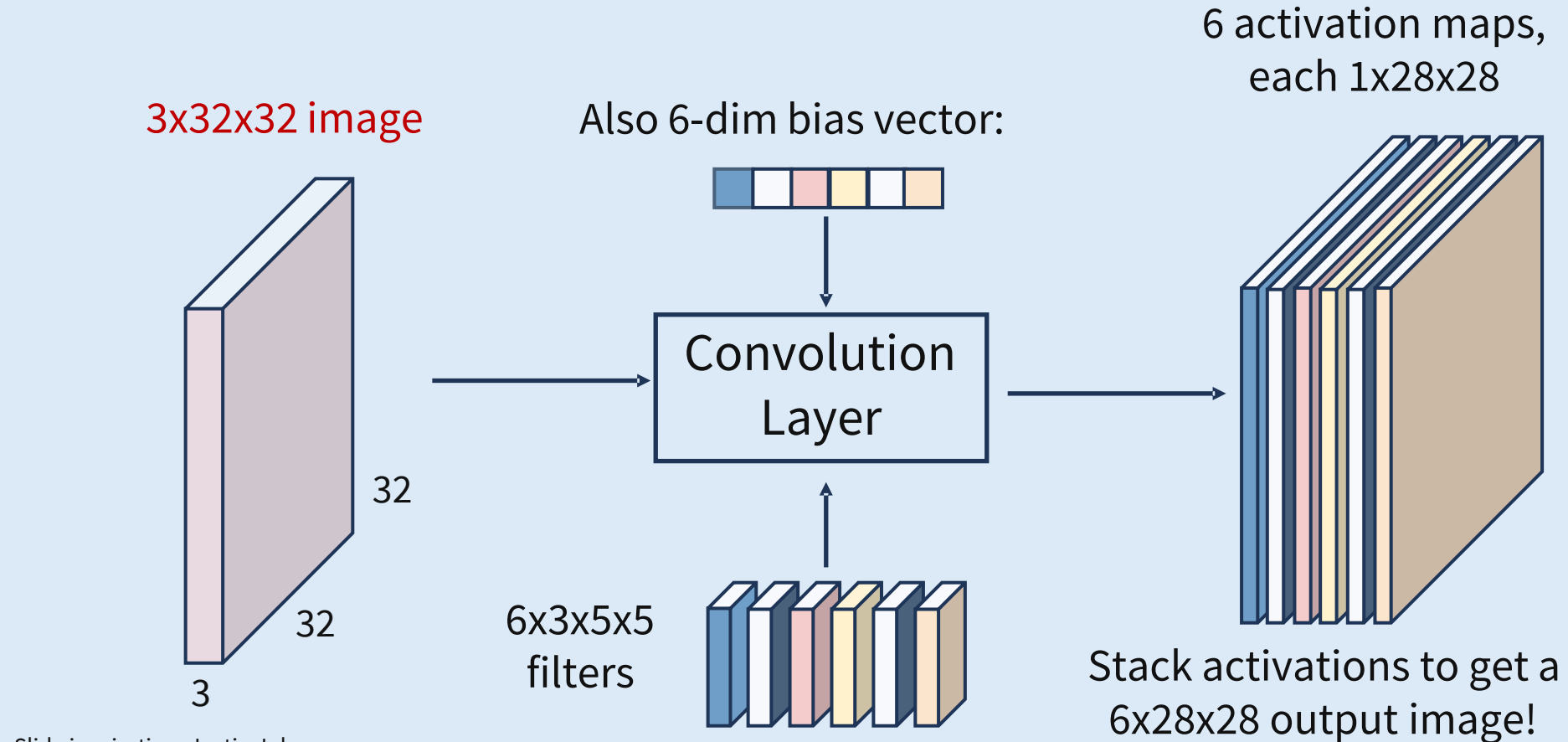


Convolutional layer



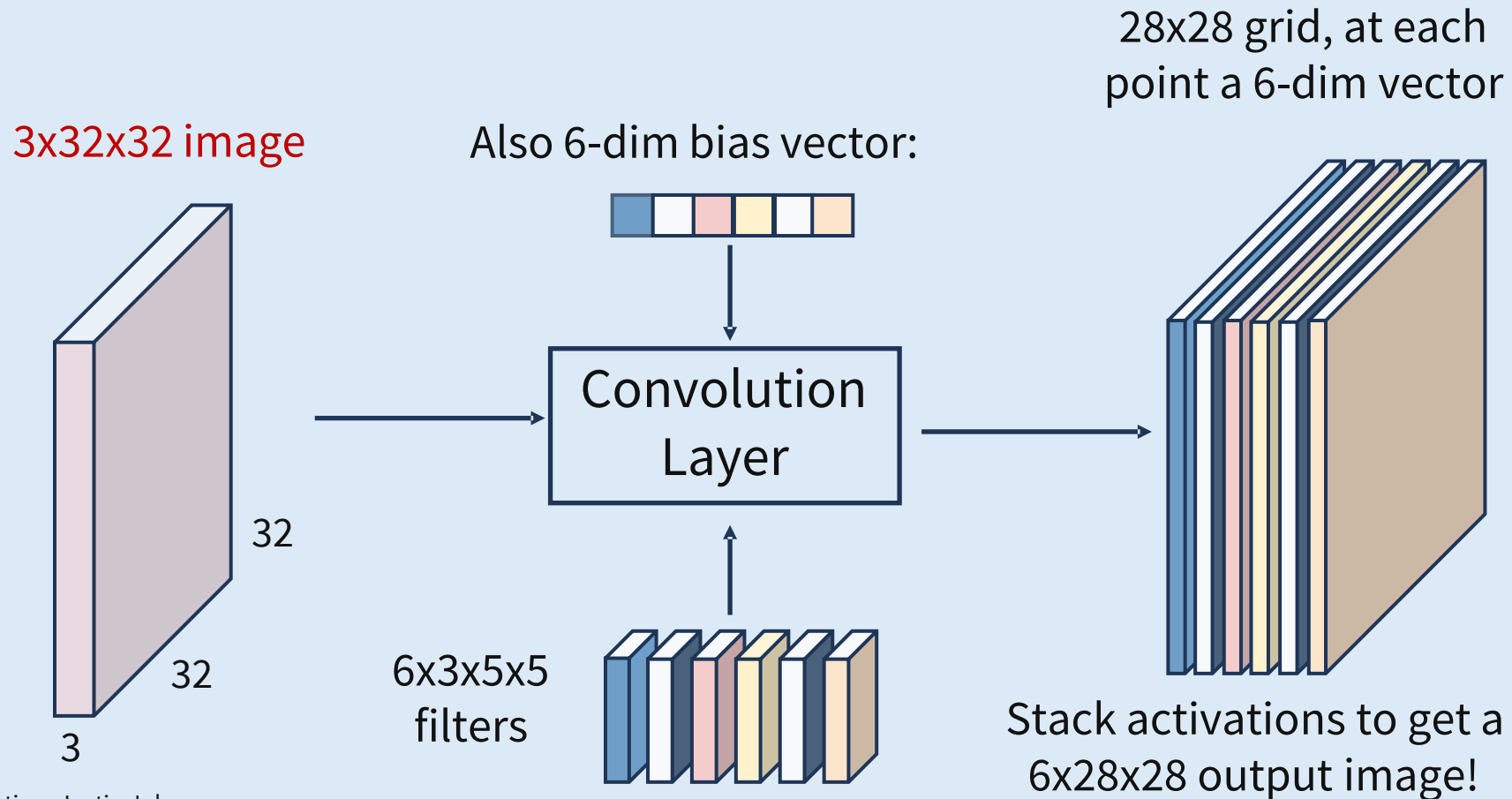
Slide inspiration: Justin Johnson

Convolutional layer



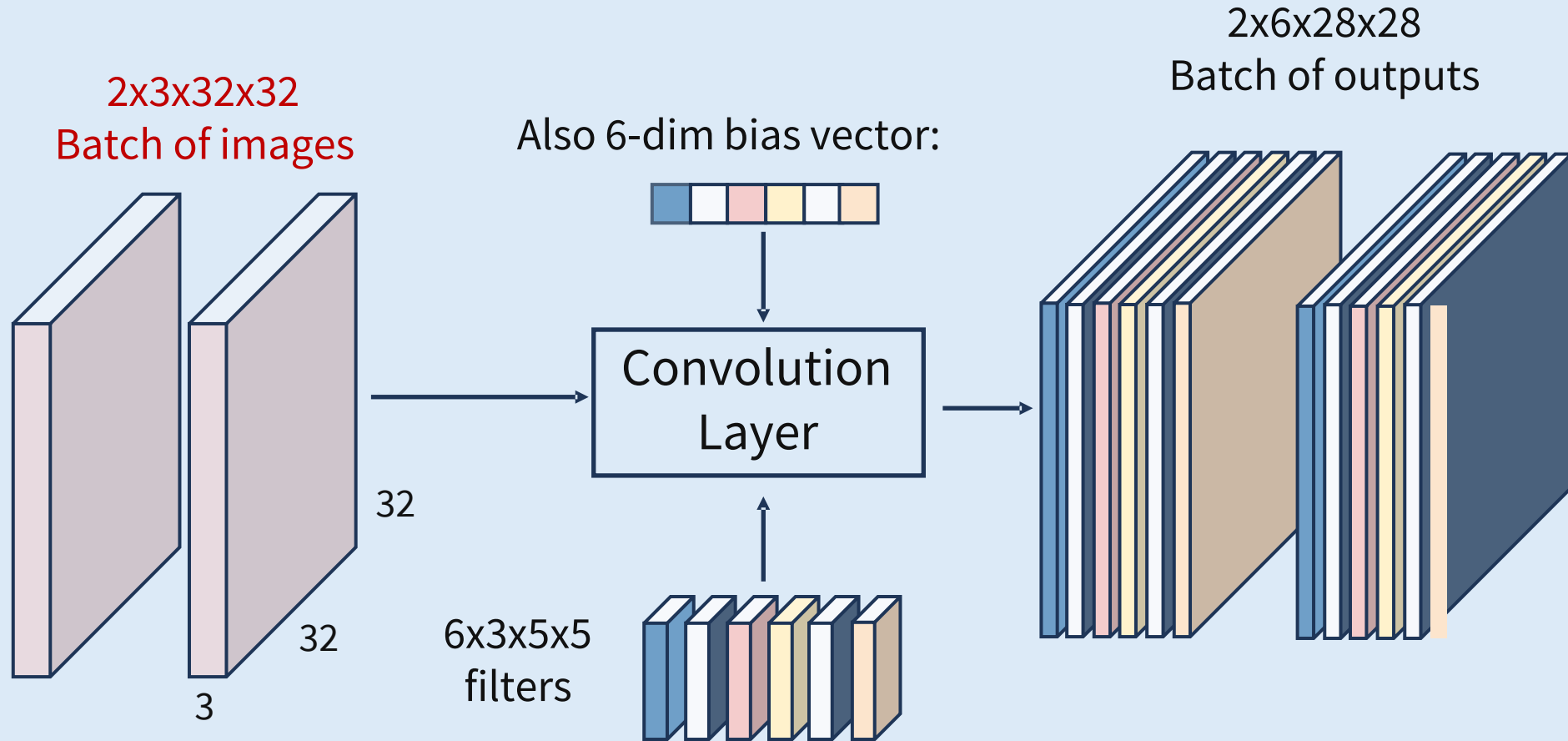
Slide inspiration: Justin Johnson

Convolutional layer



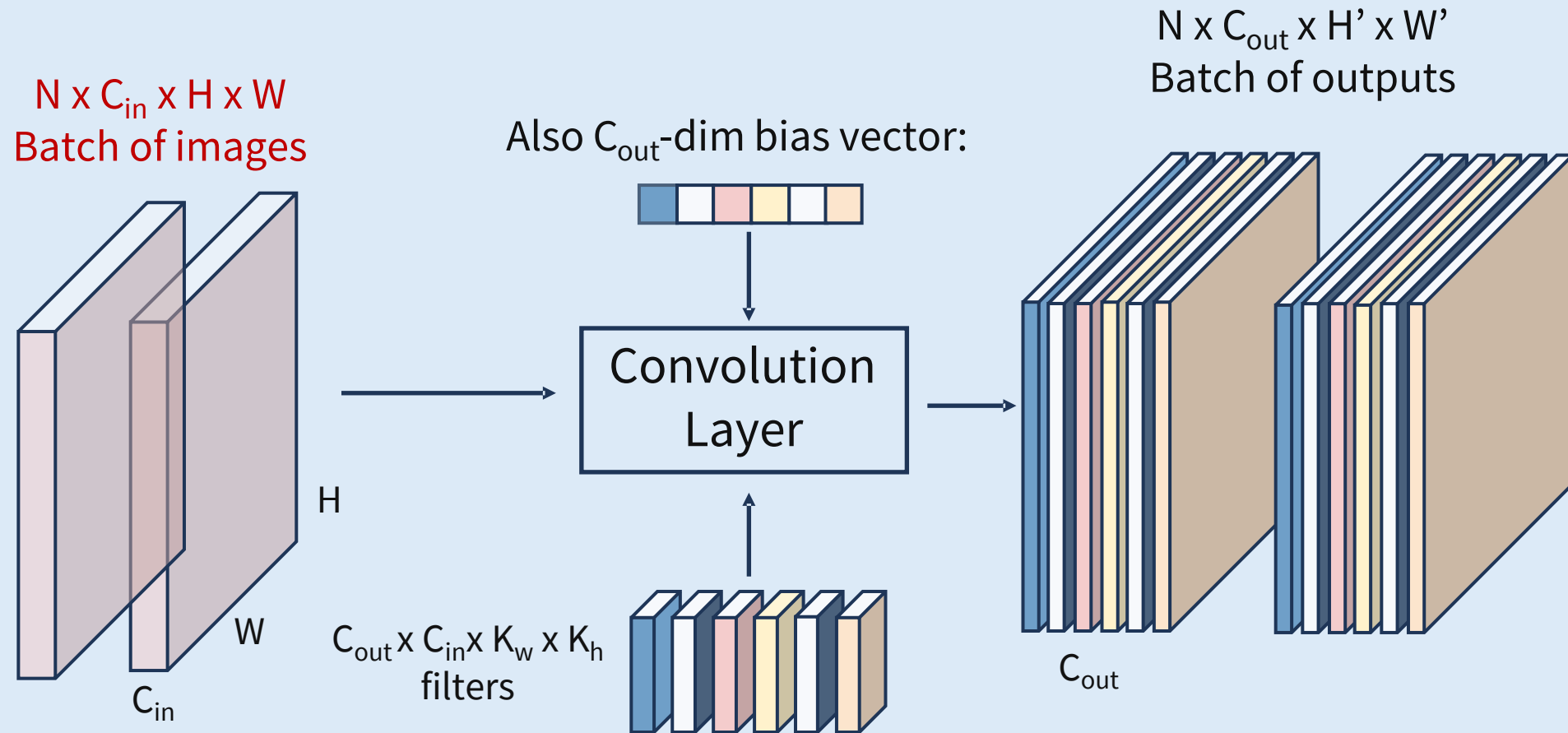
Slide inspiration: Justin Johnson

Convolutional layer



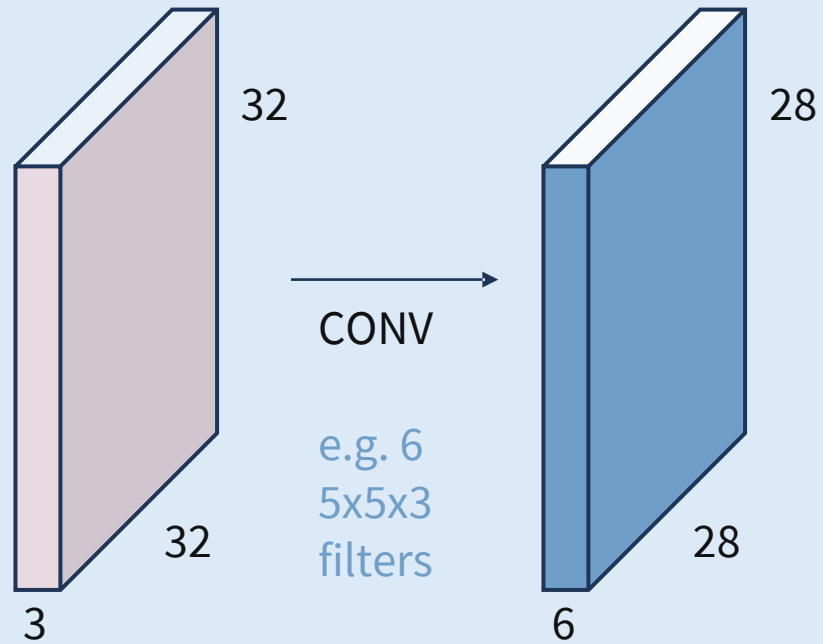
Slide inspiration: Justin Johnson

Convolutional layer

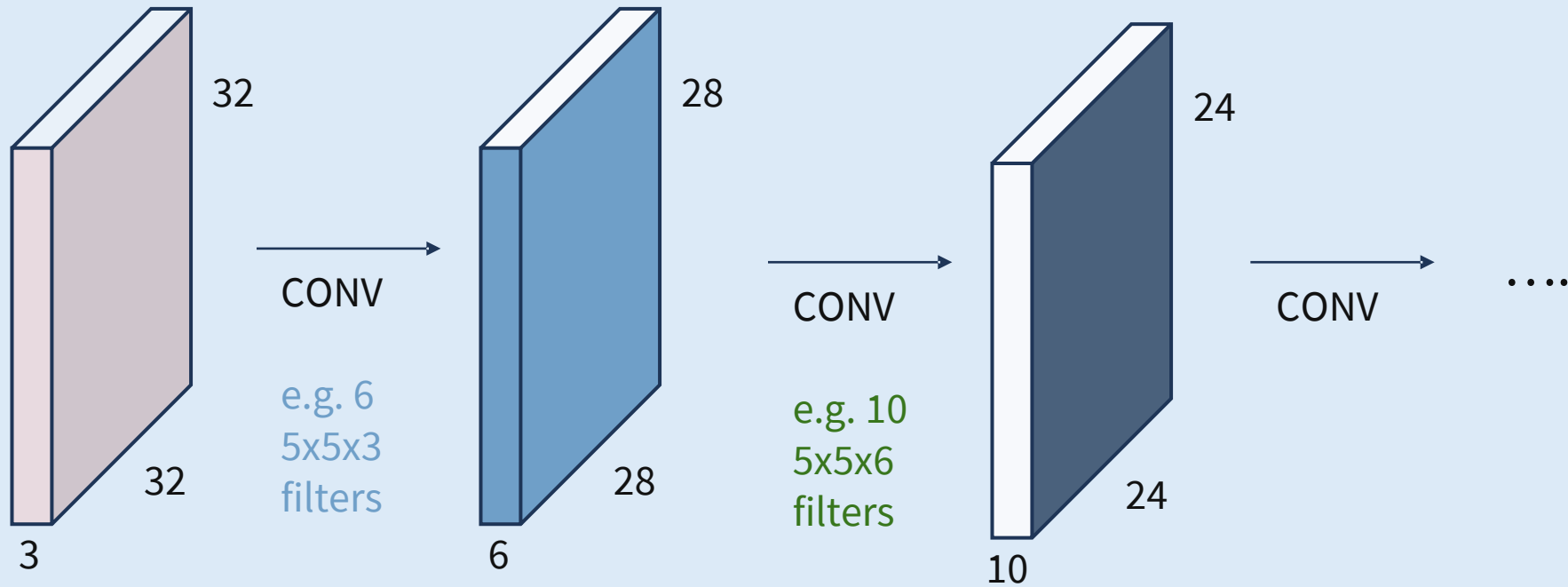


Slide inspiration: Justin Johnson

ConvNet is a neural network with convolutional layers

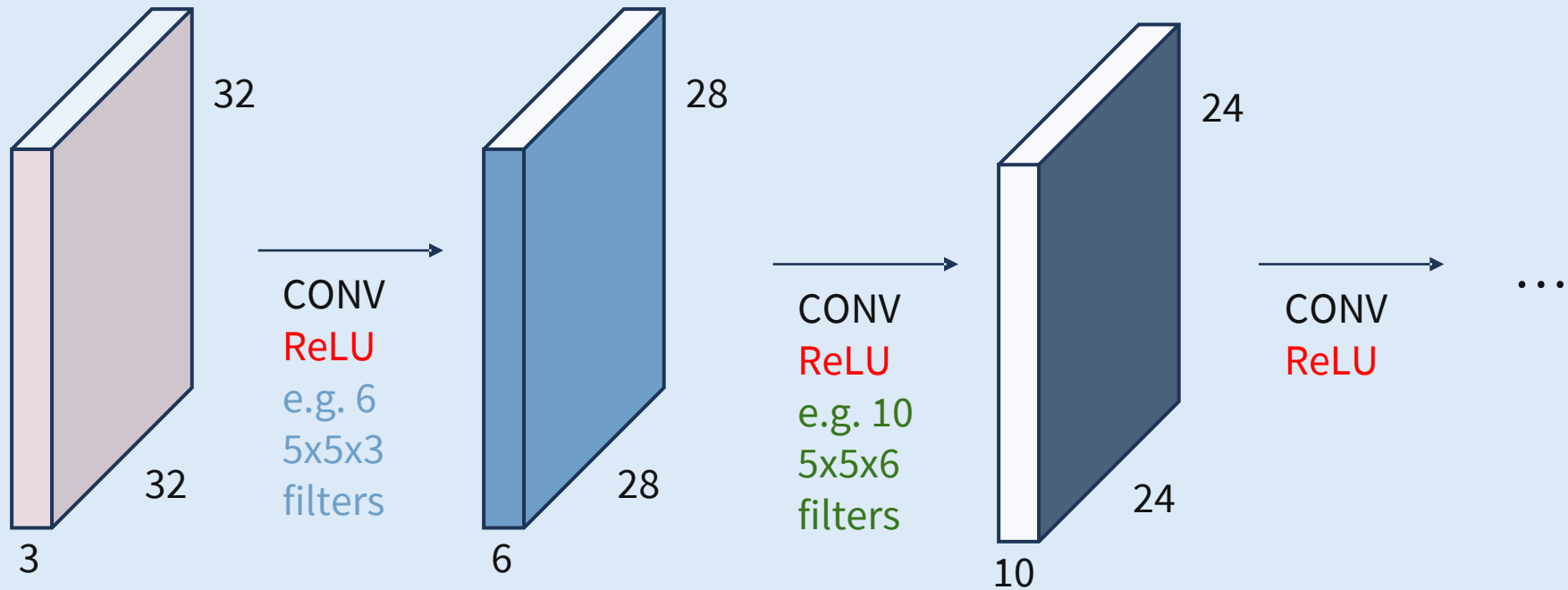


ConvNet is a neural network with convolutional layers

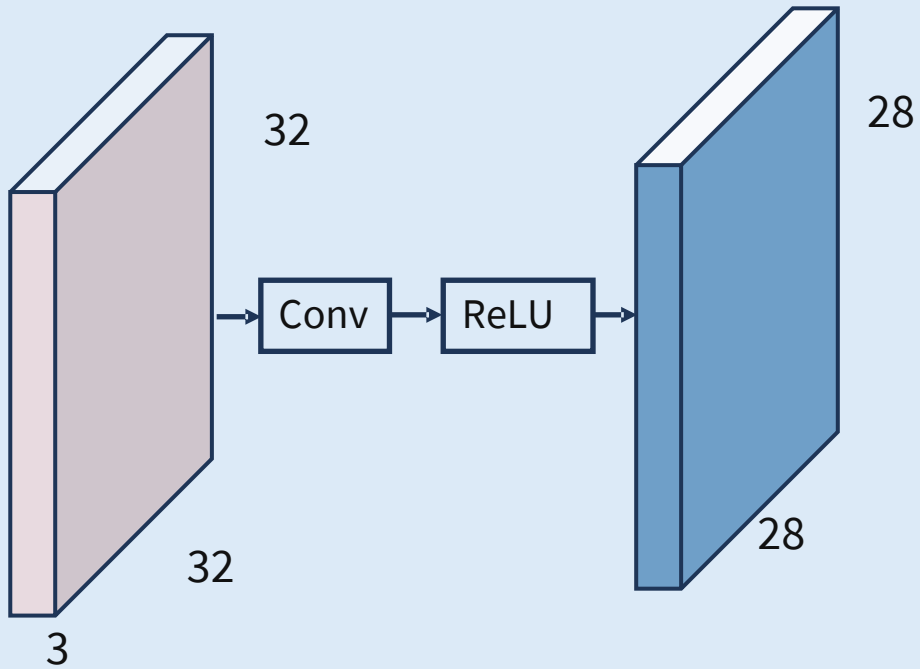


ConvNet is a neural network with convolutional layers

with activation functions!



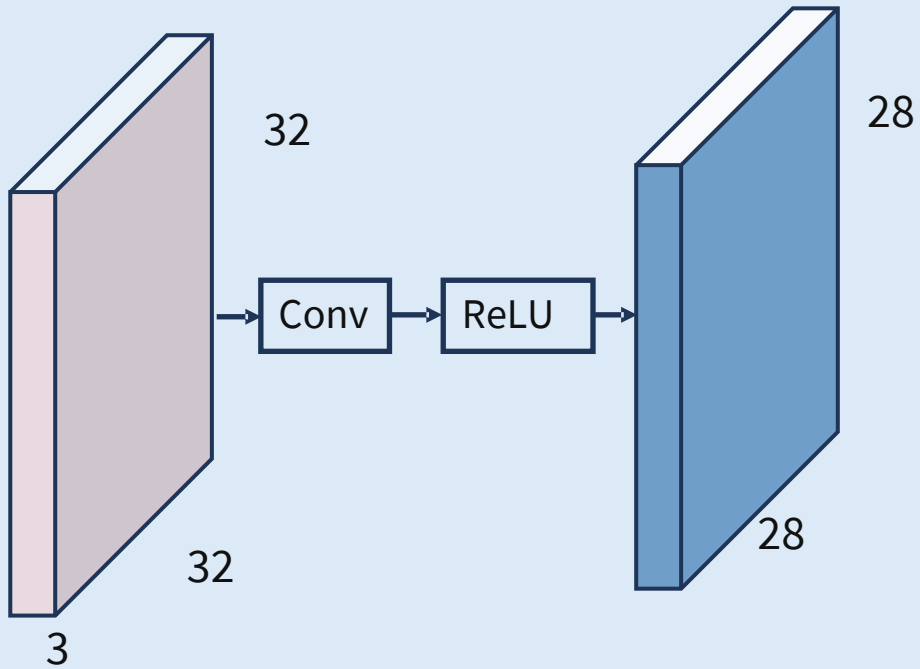
What do Conv filters learn?



Linear classifier: One template per class



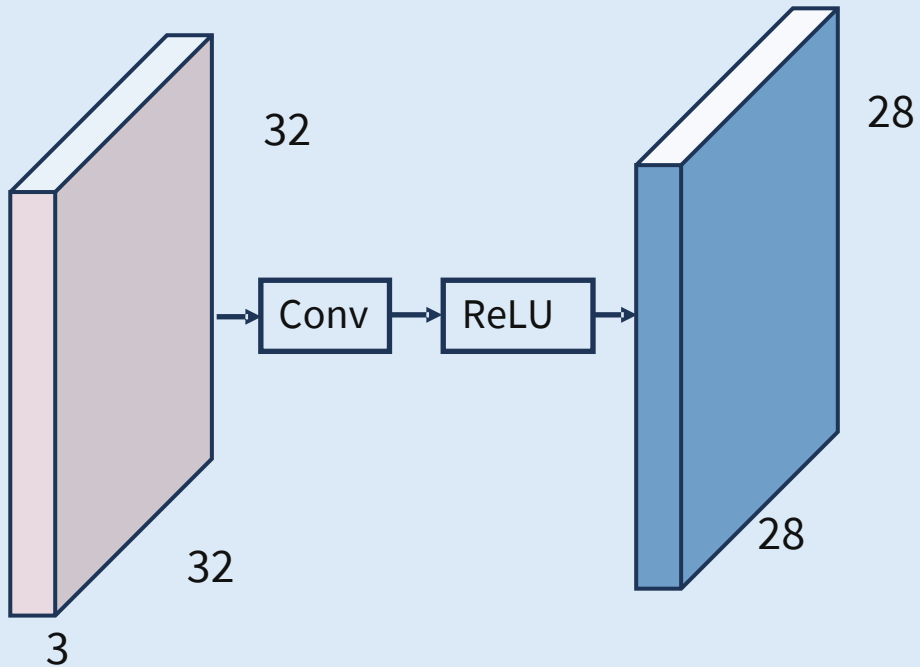
What do Conv filters learn?



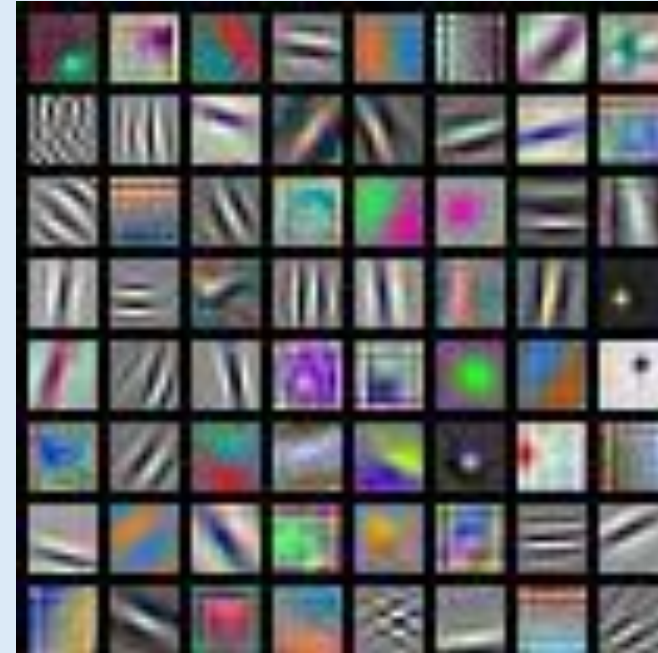
MLP: Bank of whole-image templates



What do Conv filters learn?

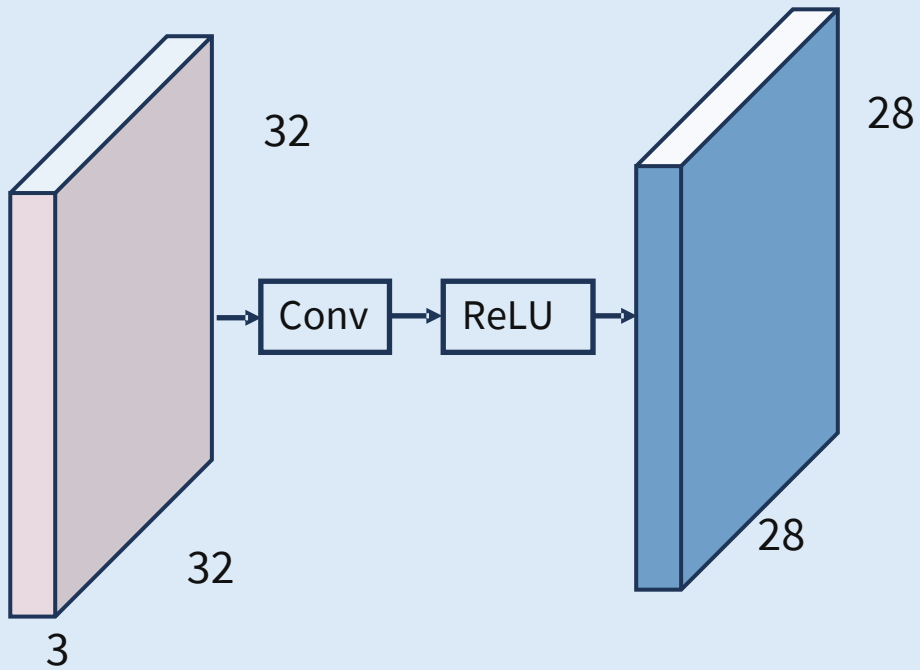


First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)

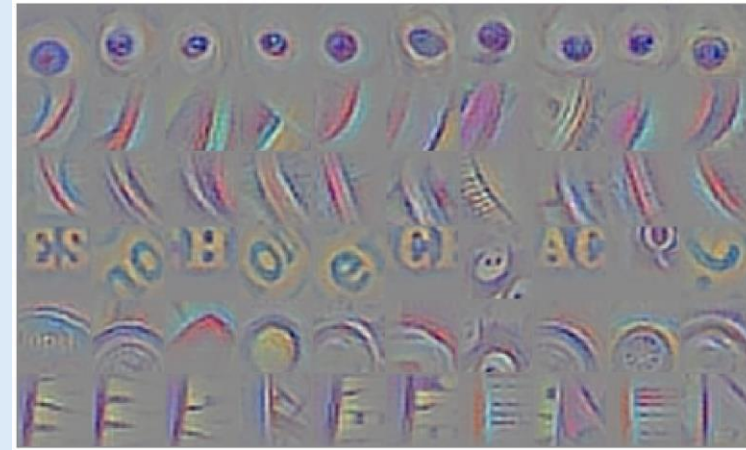


AlexNet: 64 filters, each 3x11x11

What do Conv filters learn?



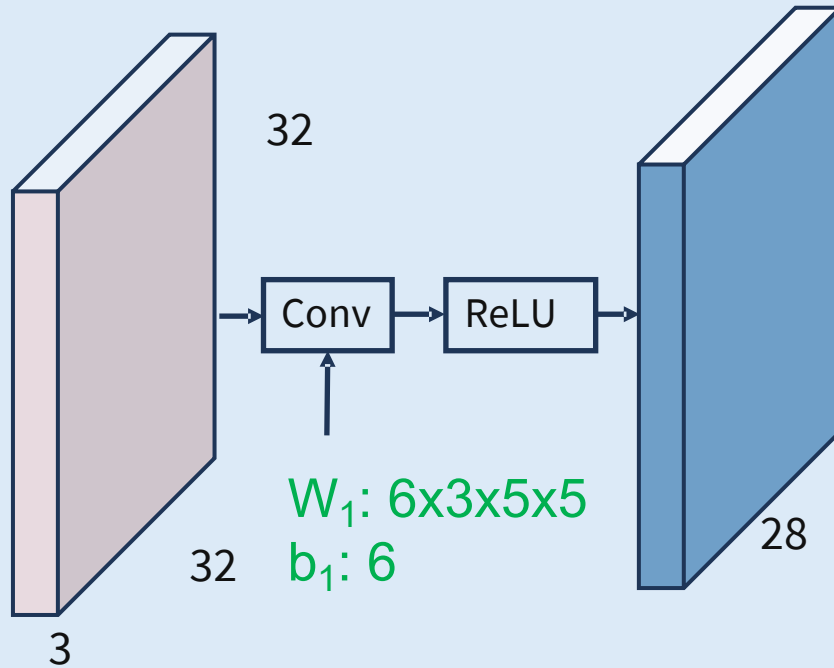
Deeper conv layers: Harder to visualize
Tend to learn larger structures e.g. eyes, letters



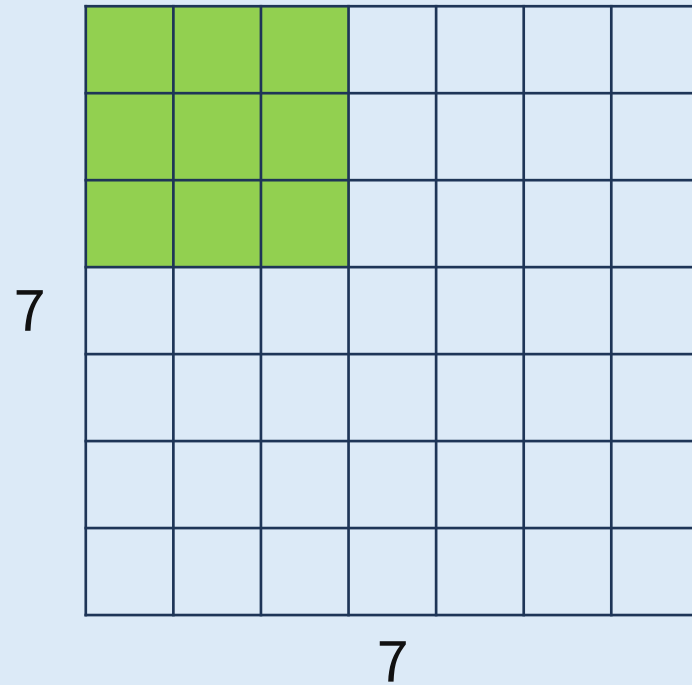
6th layer conv layer from an ImageNet model

Visualization from [Springenberg et al, ICLR 2015]

Convolution: Spatial dimensions

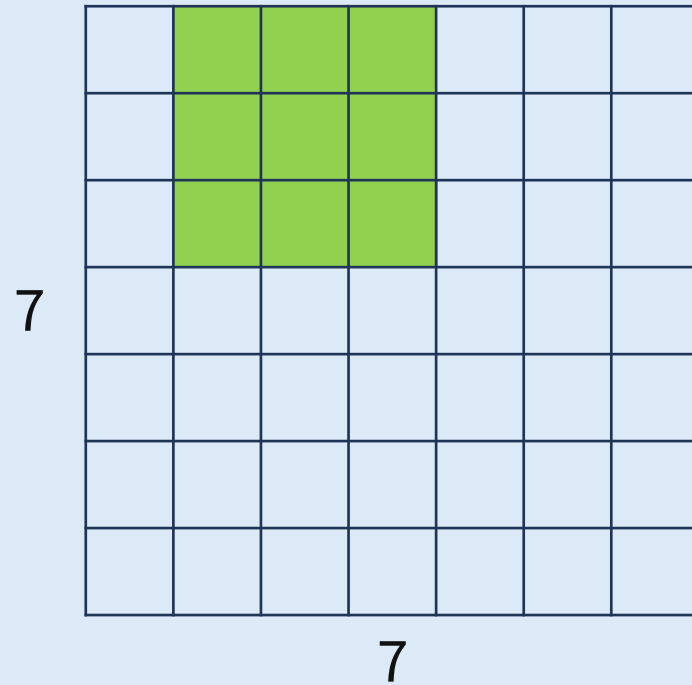


Convolution: Spatial dimensions



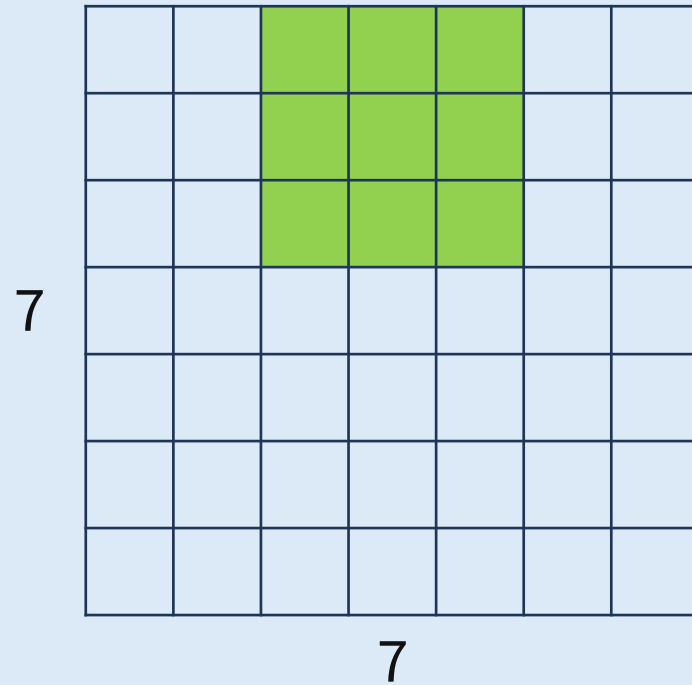
Input: 7x7
Filter: 3x3

Convolution: Spatial dimensions



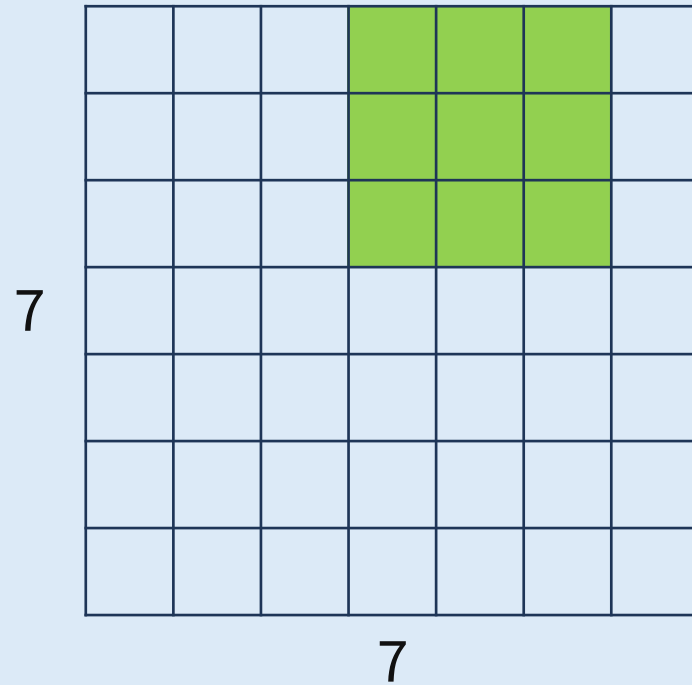
Input: 7x7
Filter: 3x3

Convolution: Spatial dimensions



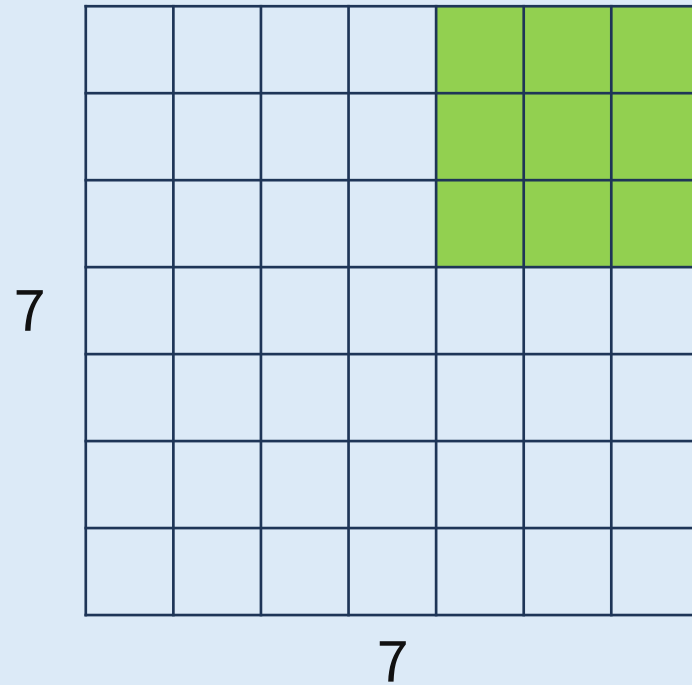
Input: 7x7
Filter: 3x3

Convolution: Spatial dimensions



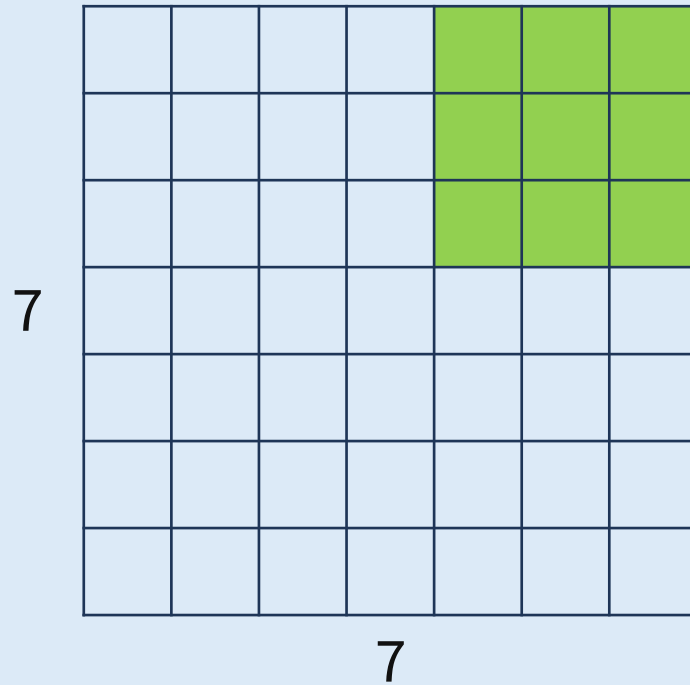
Input: 7x7
Filter: 3x3

Convolution: Spatial dimensions



Input: 7x7
Filter: 3x3

Convolution: Spatial dimensions



Input: 7x7

Filter: 3x3

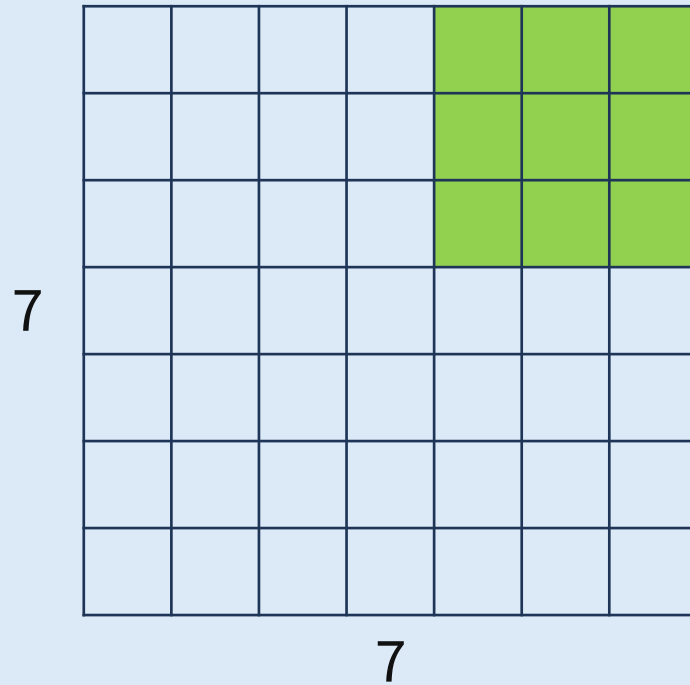
In general,

Input: W

Filter: K

Output: $W - K + 1$

Convolution: Spatial dimensions



Input: 7x7
Filter: 3x3

Problem: Feature maps shrink with each layer!

In general,
Input: W
Filter: K
Output: $W - K + 1$

Convolution: Spatial dimensions

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input: 7x7
Filter: 3x3
Output: 5x5

Problem: Feature maps shrink with each layer!

In general
Input: W
Filter: K
Padding: P

Solution: Add **padding** around the input before sliding the filter

Output: $W - K + 1 + 2P$

Convolution: Spatial dimensions

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input: 7x7
Filter: 3x3
Output: 5x5

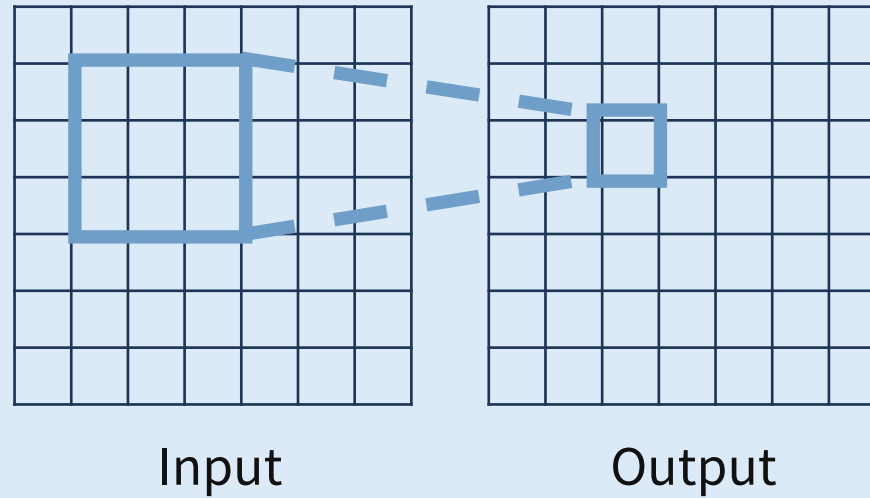
Problem: Feature maps shrink with each layer!

In general
Input: W
Filter: K
Padding: P
Output: $W - K + 1 + 2P$

Common setting:
 $P = (K - 1) / 2$
Means output has same size as input

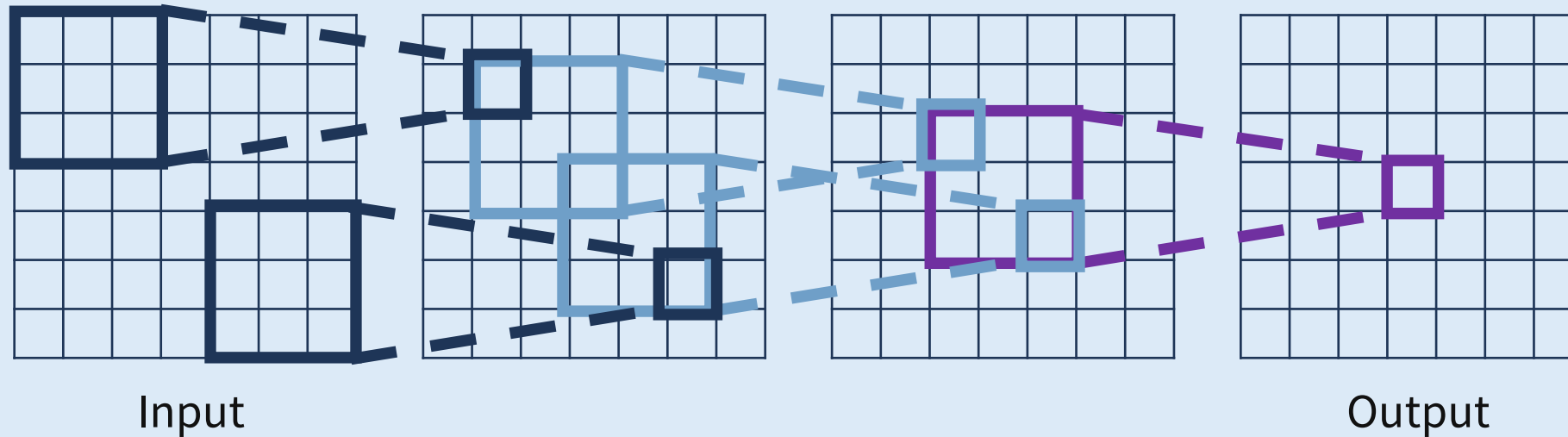
Receptive fields

For convolution with **kernel size K**, each element in the output depends on a $K \times K$ receptive field in the input



Receptive fields

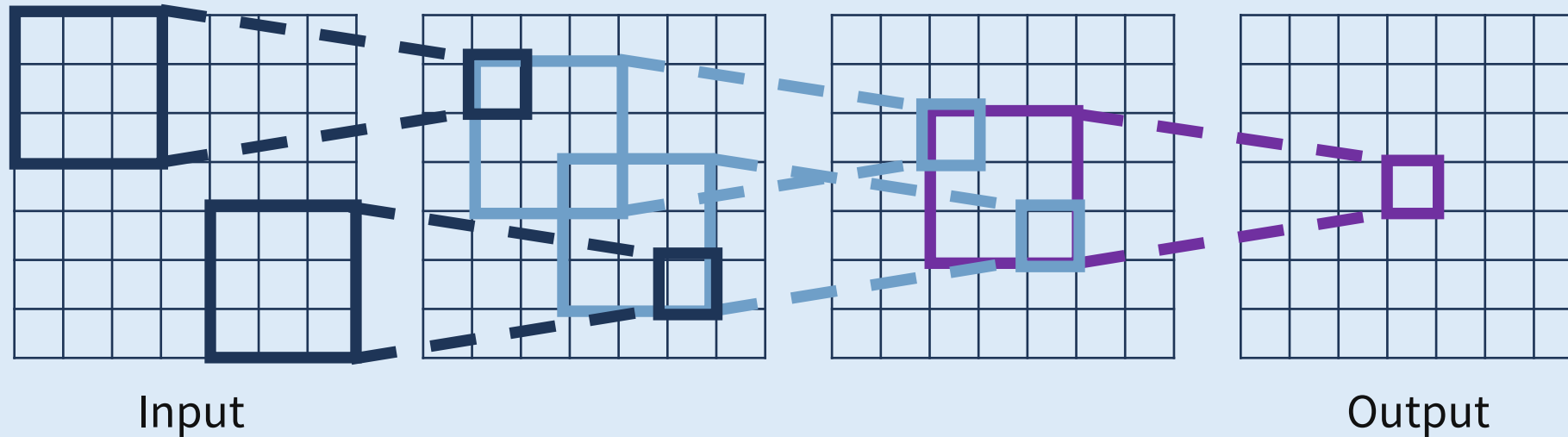
Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$



Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

Receptive fields

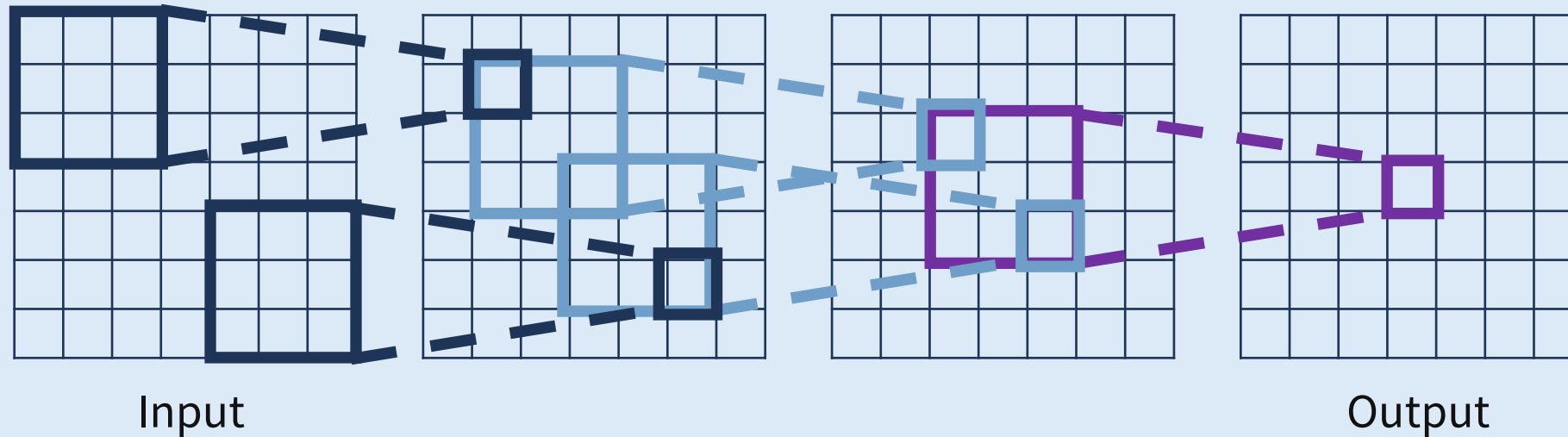
Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$



Problem: For large images we need many layers for each output to “see” the whole image

Receptive fields

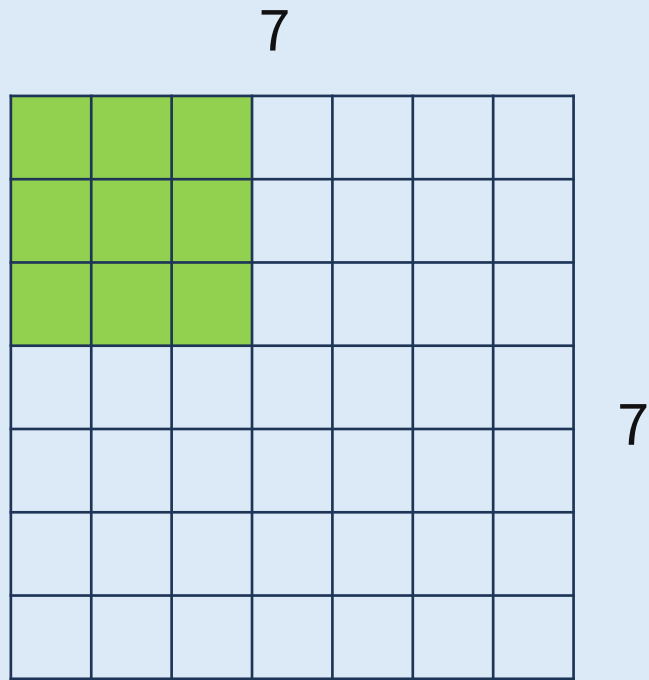
Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$



Problem: For large images we need many layers for each output to “see” the whole image

Solution: Downsample inside the network

Strided convolution

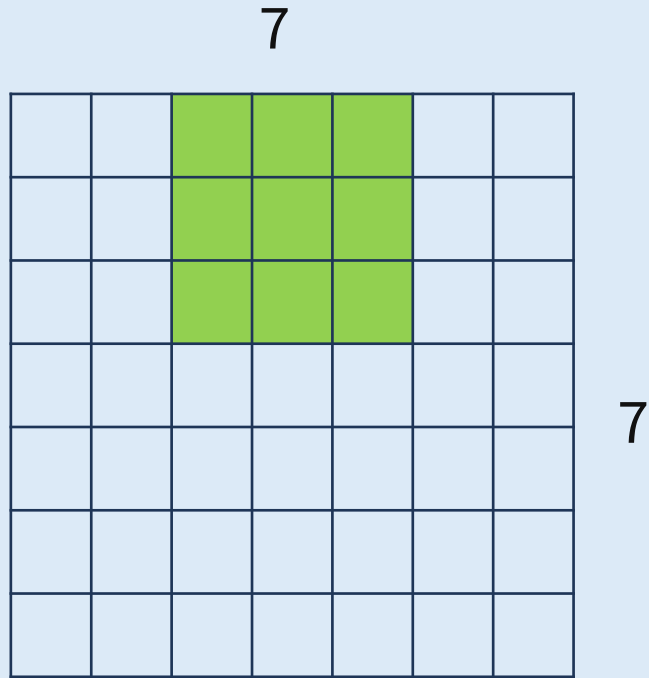


Input: 7x7

Filter: 3x3

Stride: 2

Strided convolution

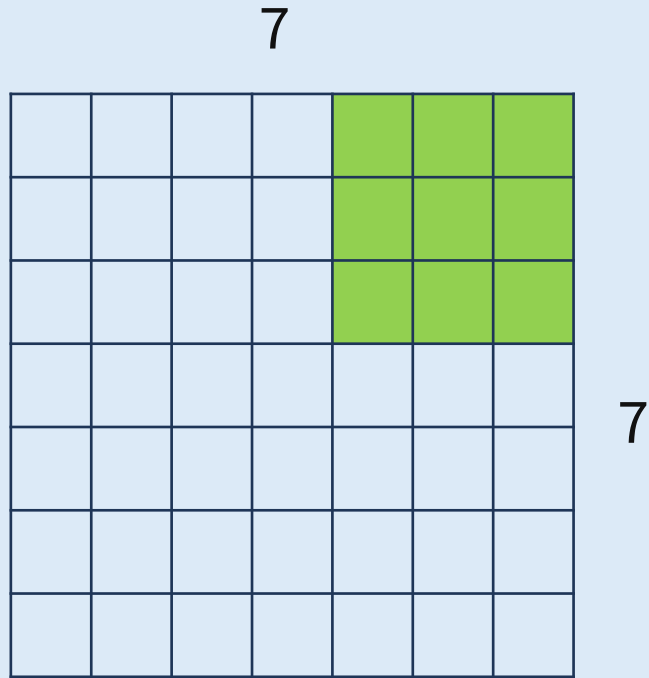


Input: 7x7

Filter: 3x3

Stride: 2

Strided convolution



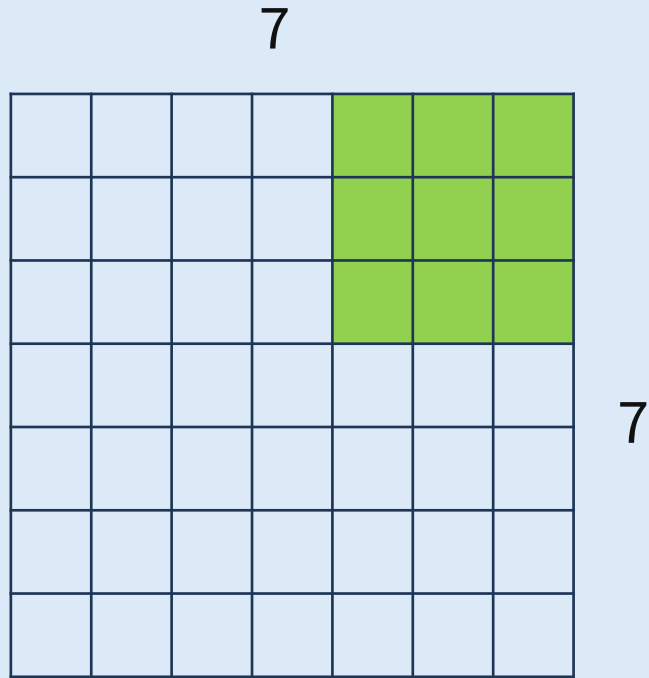
Input: 7x7

Filter: 3x3

Stride: 2

Output: 3x3

Strided convolution



Input: 7x7

Filter: 3x3

Stride: 2

Output: 3x3

In general:

Input: W

Filter: K

Padding: P

Stride: S



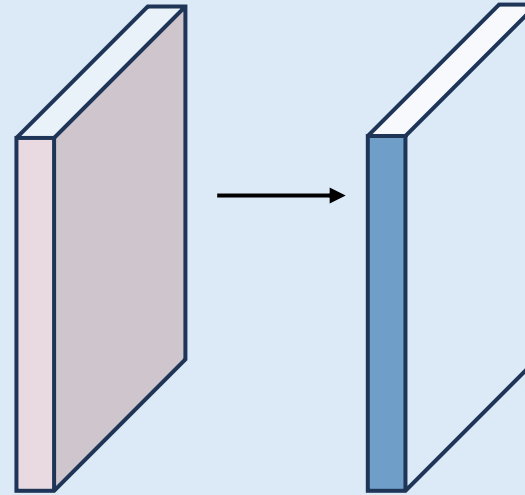
Output:

$$(W - K + 2P) / S + 1$$

Convolution example

Input volume: $3 \times 32 \times 32$
10 5×5 filters with stride 1, pad 2

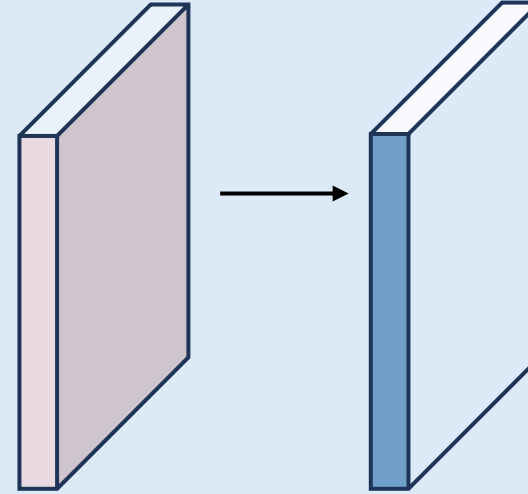
Output volume size: ?



Convolution example

Input volume: 3 x 32 x 32
10 5x5 filters with stride 1, pad 2

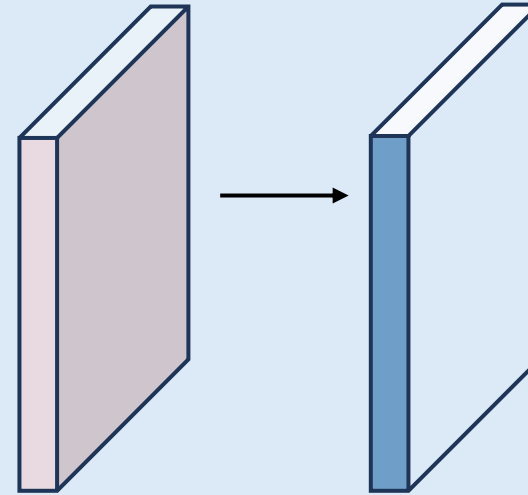
Output volume size: 10 x 32 x 32
 $32 = (32 + 2 * 2 - 5) / 1 + 1$



Convolution example

Input volume: $3 \times 32 \times 32$
10 5×5 filters with stride 1, pad 2

Output volume size: $10 \times 32 \times 32$
Number of learnable parameters: ?



Convolution example

Input volume: 3 x 32 x 32

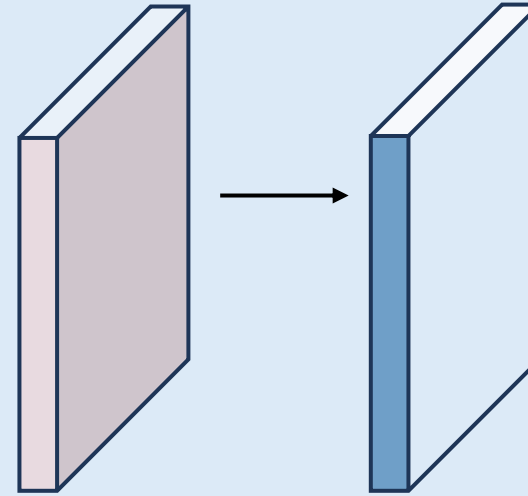
10 5x5 filters with stride 1, pad 2

Output volume size: 10 x 32 x 32

Number of learnable parameters: 760

Parameters per filter: $3 * 5 * 5 + 1$ (for bias) = 76

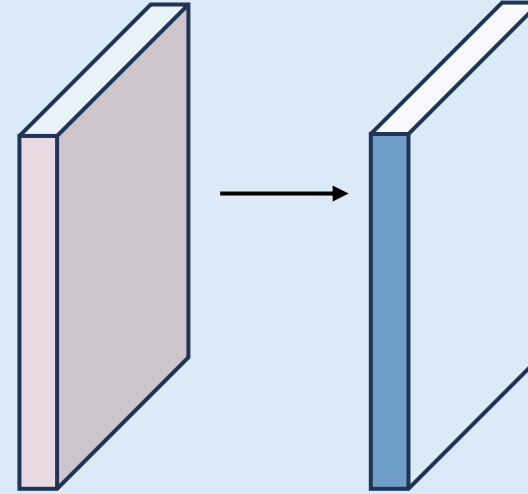
10 filters, so total is $10 * 76 = 760$



Convolution example

Input volume: $3 \times 32 \times 32$
10 5×5 filters with stride 1, pad 2

Output volume size: $10 \times 32 \times 32$
Number of learnable parameters: 760
Number of multiply-add operations?



Convolution example

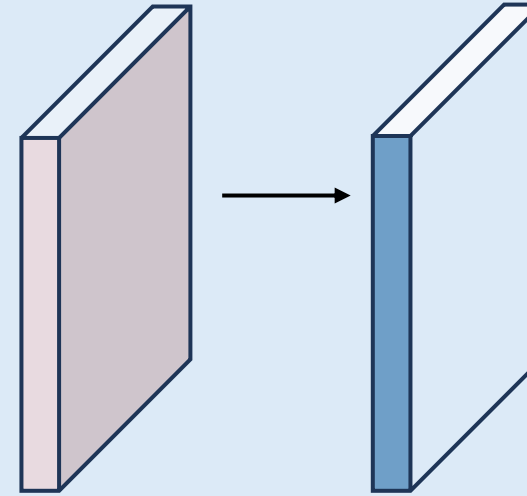
Input volume: **3** x 32 x 32
10 **5x5** filters with stride 1, pad 2

Output volume size: **10 x 32 x 32**
Number of learnable parameters: 760
Number of multiply-add operations: **768,000**

10*32*32 = 10,240 outputs

Each output is the inner product of two **3x5x5** tensors (75 elems)

Total = 75*10240 = **768K**



Convolution summary

Input: $C_{in} \times H \times W$

Hyperparameters:

- **Kernel size:** $K_H \times K_W$
- **Number filters:** C_{out}
- **Padding:** P
- **Stride:** S

Weight matrix: $C_{out} \times C_{in} \times K_H \times K_W$
giving C_{out} filters of size $C_{in} \times K_H \times K_W$

Bias vector: C_{out}

Output size: $C_{out} \times H' \times W'$ where:

- $H' = (H - K + 2P) / S + 1$
- $W' = (W - K + 2P) / S + 1$

Common settings:

$K_H = K_W$ (Small square filters)

$P = (K - 1) / 2$ ("Same" padding)

$C_{in}, C_{out} = 32, 64, 128, 256$ (powers of 2)

$K = 3, P = 1, S = 1$ (3x3 conv)

$K = 5, P = 2, S = 1$ (5x5 conv)

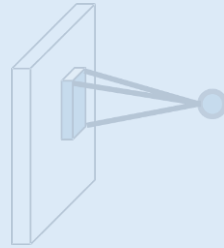
$K = 1, P = 0, S = 1$ (1x1 conv)

$K = 3, P = 1, S = 2$ (Downsample by 2)

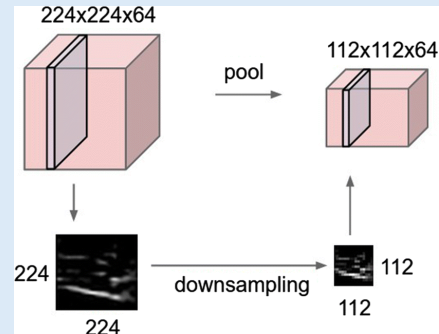
Convolutional Neural Networks (CNNs)

Today, we will study

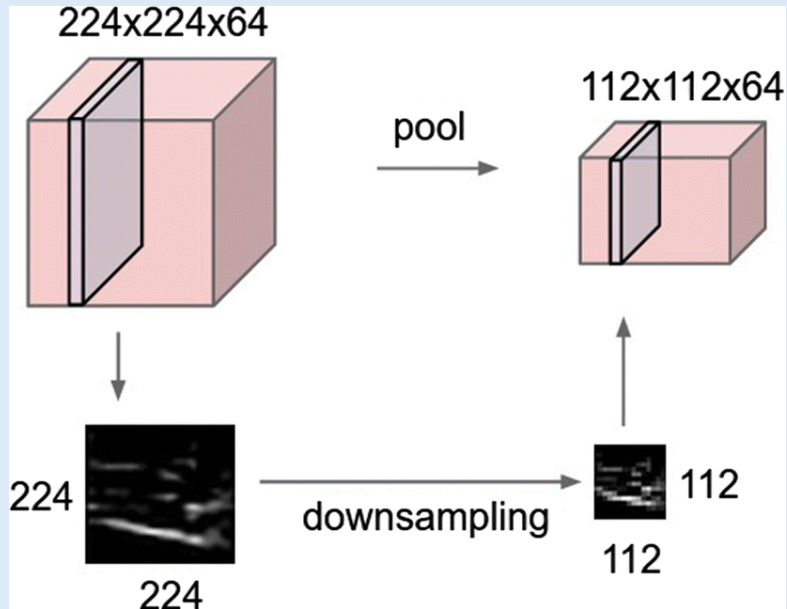
- Convolutional layer:



- Pooling layer:



Pooling Layers: Another way to downsample



Given an input $C \times H \times W$,
downsample each $1 \times H \times W$ plane

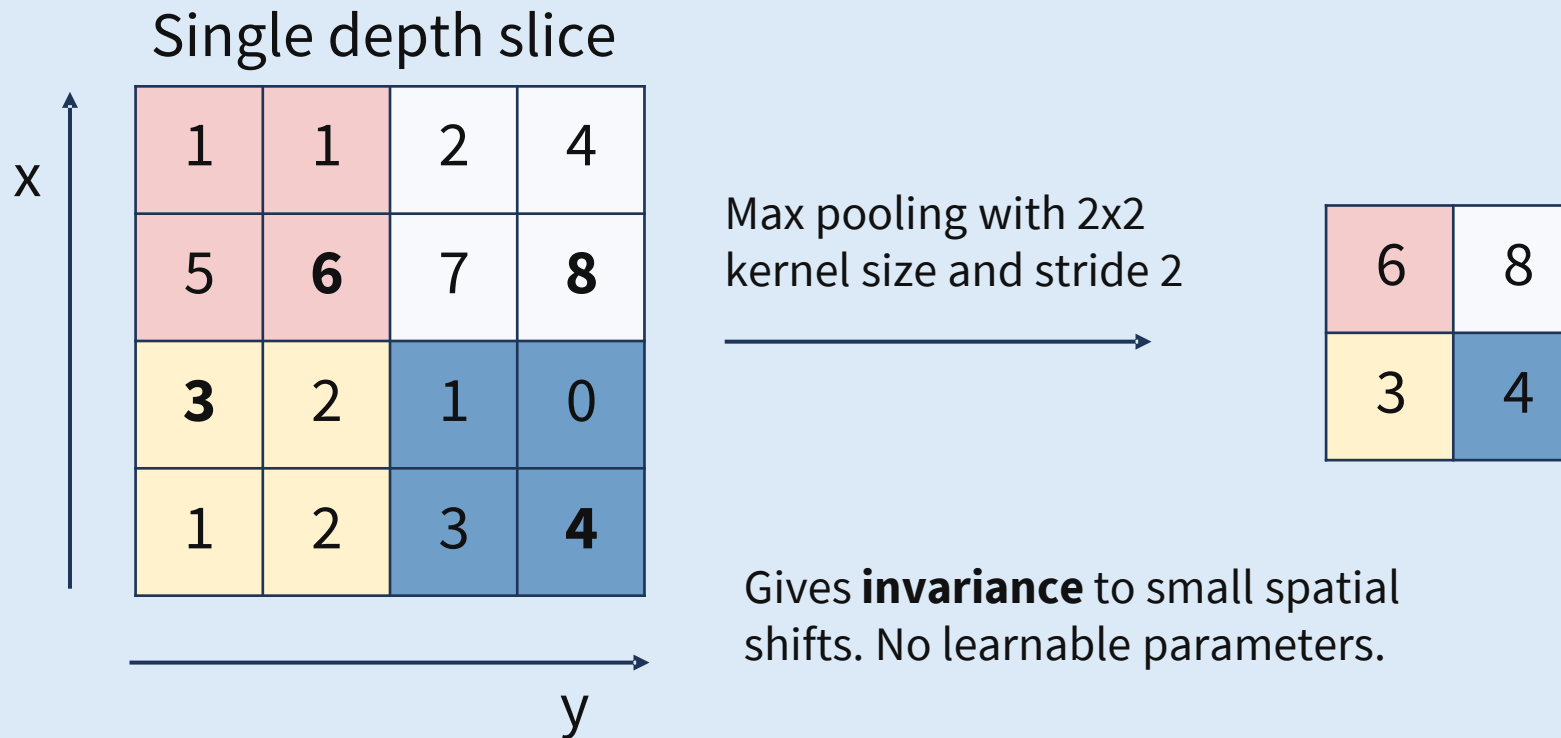
Hyperparameters:

Kernel Size

Stride

Pooling function

Pooling Layers: Another way to downsample



Pooling layers summary

Input: $C \times H \times W$

Hyperparameters:

- **Kernel size:** K
- **Stride:** S
- **Pooling function:** max, avg

Output size: $C \times H' \times W'$ where:

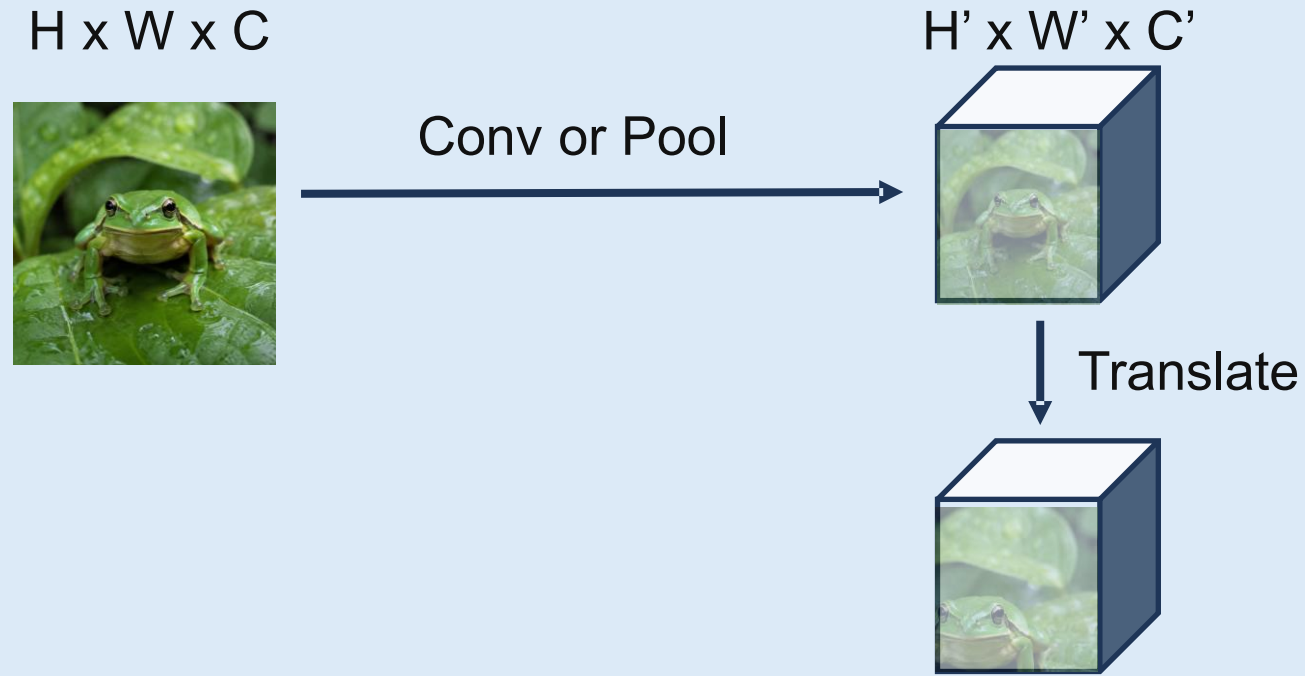
- $H' = (H - K) / S + 1$
- $W' = (W - K) / S + 1$

No learnable parameters

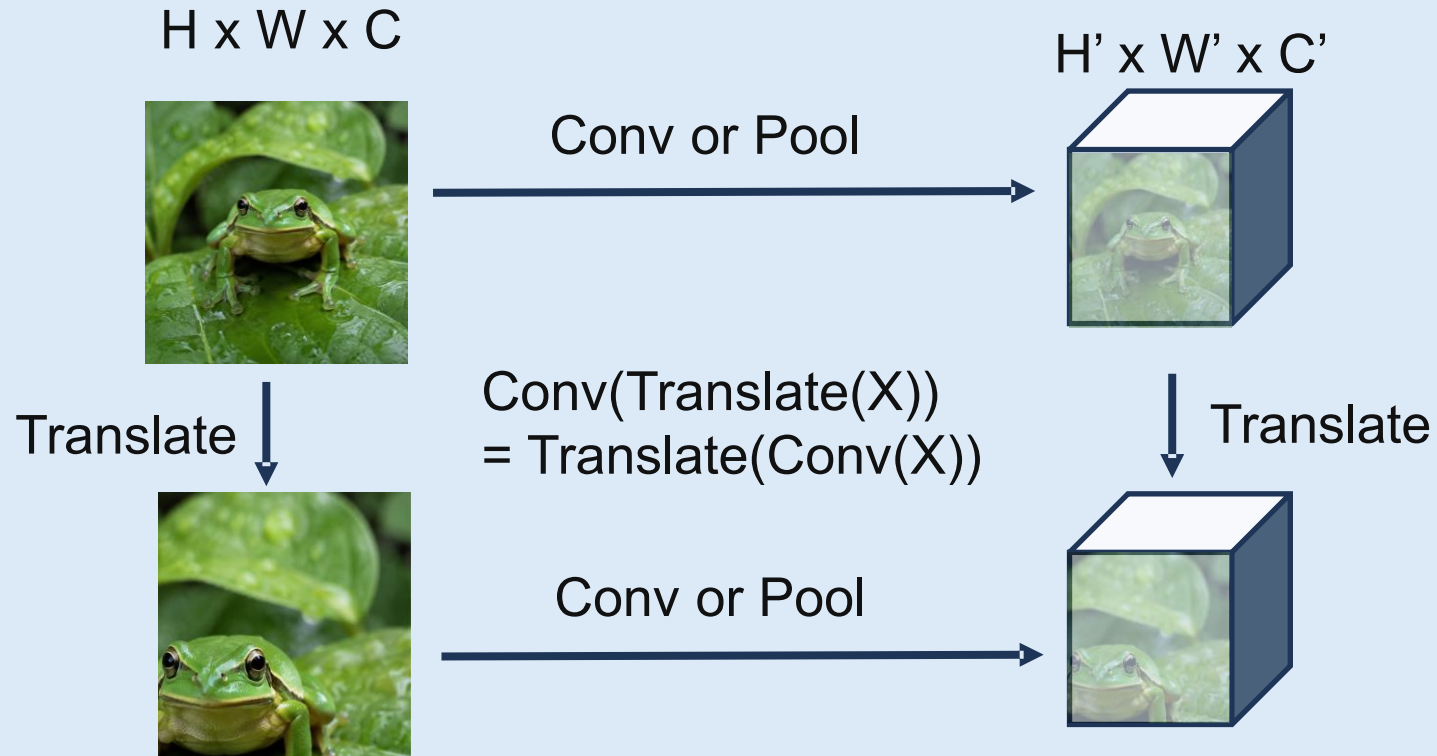
Common setting:

max, $K=2$, $S=2 \Rightarrow$ Gives 2x downsampling

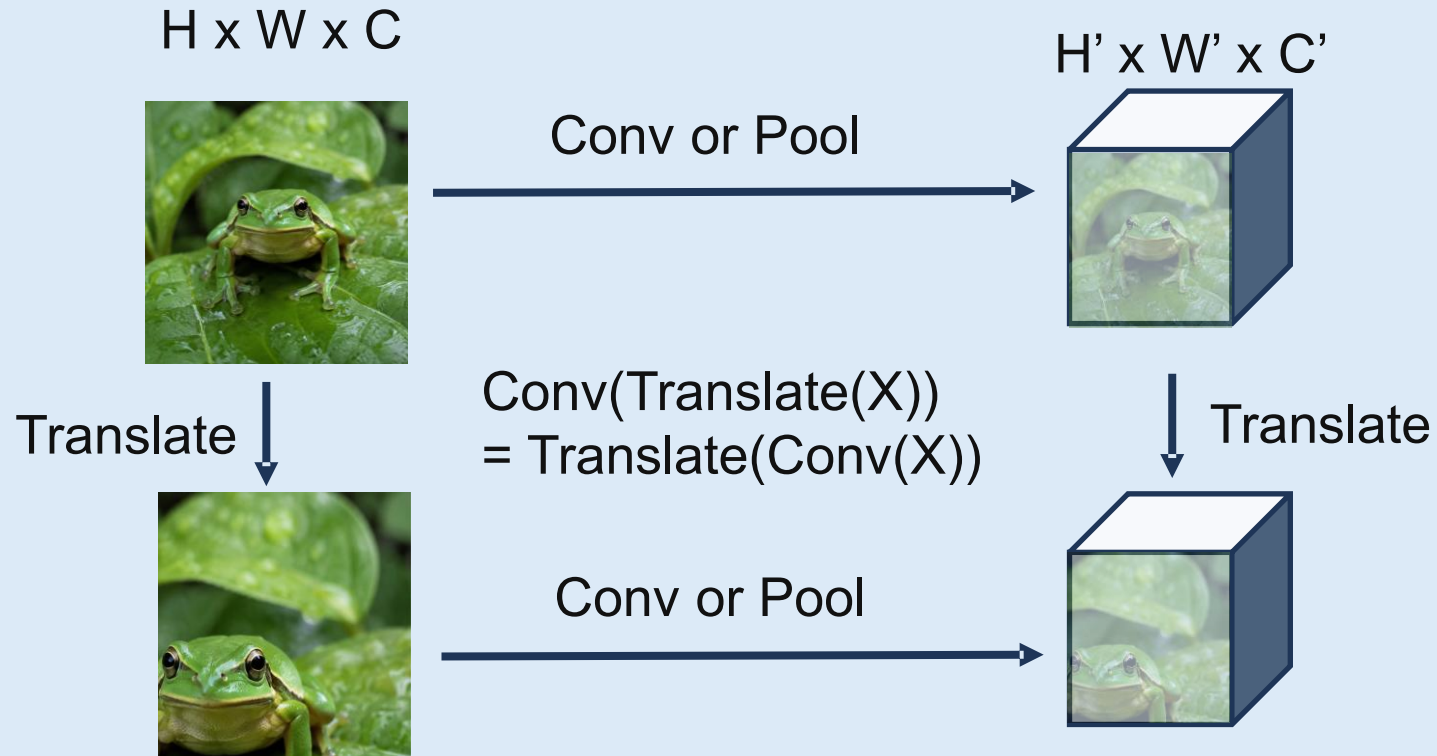
Convolution and Pooling: Translation Equivariance



Convolution and Pooling: Translation Equivariance



Convolution and Pooling: Translation Equivariance



Intuition: Features of images don't depend on their location in the image

References

- Stanford CS231n
- Mathematical Foundations of Deep Neural Networks at Seoul National University.

